

스크린

강사

정준모

조용민

김민규

하지훈

한동훈

차재민

최현서

차희성

김정훈

박준재

방현수

김정찬

장원우

김형규

김현수

김진실

김미성

최재희

김가은

정석환

이주연

박재홍

배성우

임태현

최민호

이서범

오희진

보조강사

최인영

조재경

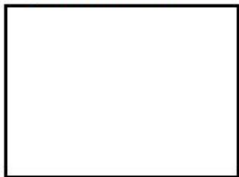
주동민

7.20.(월)

가상환경

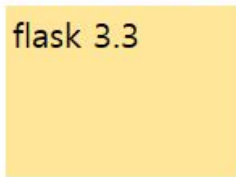
PC

Python 3.14



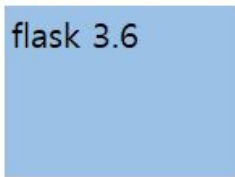
가상환경 1 - a

flask 3.3



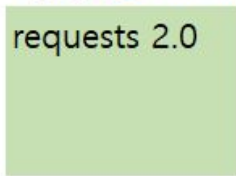
가상환경 2 - b

flask 3.6



가상환경 3 - .venv

requests 2.0



010-8478-8182 김남현

seorab@kakao.com

파이썬 설치 후 해야 할 작업

1. 가상환경 생성
2. 가상환경 활성화
3. 사용할 도구 설치 (pip install 도구명)
 - pip install ipykernel

개발 환경 설정

1. 파이썬 설치

2. VS Code 설치 + Python 확장 프로그램 설치

3. 가상환경 생성

→ `python -m venv` 가상환경명

ex) `python -m venv .venv`

4. 가상환경 활성화

→ `call .venv/Scripts/Activate`

주피터 노트북 파일 생성

1. (VS Code 에서)

명령 팔레트(Shift + Ctrl + P)에서 create jupyter 검색

2. 파일 저장

(저장하지 않으면 파일이 생성되지 않음)

네이밍 컨벤션 Naming Convention

1. 카멜 camel case : person age → personAge
2. 파스칼 pascal case : person age → PersonAge
3. 스네이크 snake case : person age → person_age
4. 케밥 kebab case : person age → person-age

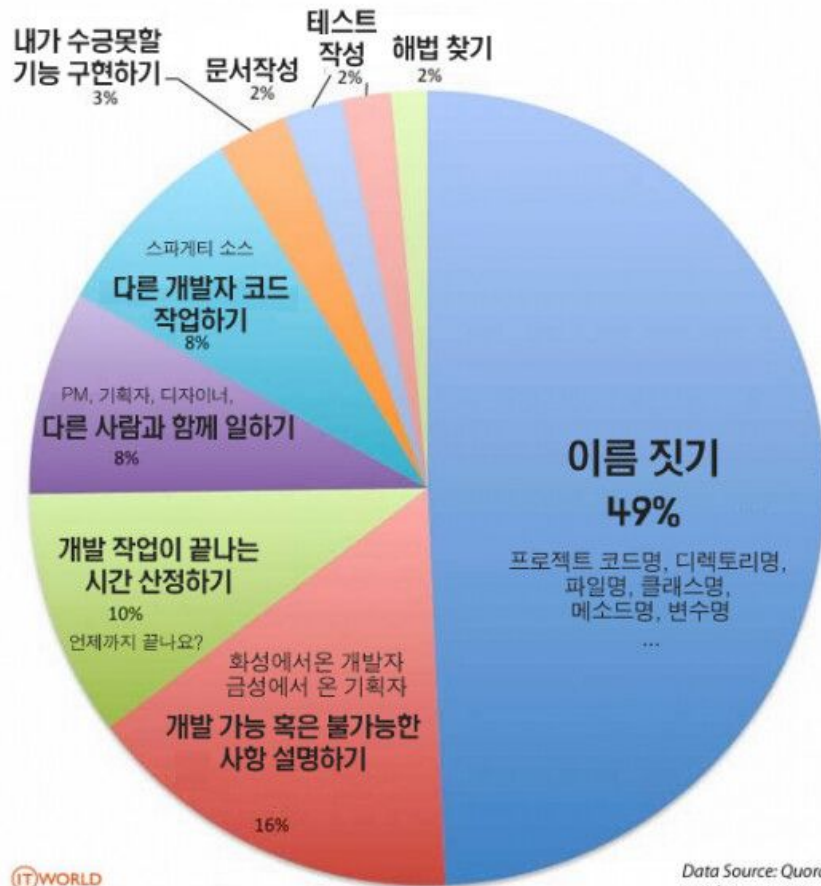
변수

값을 저장하기 위한 공간

복잡하고 긴 값을 미리 기억해놓고 재사용

기존의 값을 계속 활용

프로그래머가 가장 힘들어하는 일은?



변수명 짓기

변수명 짓기

<https://www.curiousstore.com/#!/util/naming>

숫자 자료형

정수, 실수, 2/8/16진수

1. 사칙연산(+ - * /), 몫(`//`), 나머지(`%`), 지수(`**`)

2. 다른 형태의 자료를 숫자로 변환

- 정수로 변환 → `int(다른 형태의 자료)`

- 실수로 변환 → `float(다른 형태의 자료)`

연습문제

초를 '00시 00분 00초' 형태로 변환하여 출력하는 프로그램을 만들려고 합니다.
몫 연산자(//)와 나머지 연산자(%), 그리고 f-string 포매팅을 활용하여
작성해보세요.

```
# 전체 시간 (3703초)
```

```
sec = 3703
```

```
# print(f'{hour}시간 {minute}분 {second}초') → 1시간 1분 43초
```

페이징 공식

1. 페이지 이동을 위한 네비게이션 번호를 계산하는 식

1page $\rightarrow$ 1 ~ 10

7page $\rightarrow$ 1 ~ 10

16page $\rightarrow$ 11 ~ 20

첫번째 페이지 번호 : $(\text{현재 페이지 번호} - 1) // 10 * 10 + 1$

마지막 데이터 번호 : 첫번째 페이지 번호 + 9

페이징 공식

2. 조회할 데이터 번호를 계산하는 식

1page $\rightarrow$ 1 ~ 10

2page $\rightarrow$ 11 ~ 20

10page $\rightarrow$ 91 ~ 100

마지막 데이터 번호 : 현재 페이지 번호 * 10

첫번째 데이터 번호 : 마지막 데이터 번호 - 9

문자 자료형

작은 따옴표 또는 큰 따옴표 사용 **ex)** ' " '""

따옴표 1개 또는 3개

1. Indexing (해당 위치의 문자 1개 조회 - 문자 한개 추출)
 2. Slicing (지정된 범위의 문자 N개 조회 - 문자 여러개 잘라내기)
 3. Formatting (문자열의 틀(템플릿)을 잡고 가변적인 값만 {} 안에 쏙 꽂아 재사용)
3. 문자 관련 함수

문자 자료형 연습문제

<http://ggoreb.com/quiz/소나기.txt>

- "소나기" 개수 확인 → 2개

- "첩" 으로 시작하는 문장 찾기 → 첩덩굴이 엉키어 꽃을 달고 있었다.

7.21.(화)

암호화 문자열

t = "g fmnc wms bgblr rpylqjyrc gr zw fylb. rfyrq ufyr amknsrcpq ypc
dmp. bmgle gr gl zw fylb gq glcddgagclr ylb rfyr'q ufw rfgq rcvr gq qm
jmle. sqgle qrpgle.kyicrpylq() gq pcamkkclbcb. lmu ynnjw ml rfc spj."

before = 'abcdefghijklmnopqrstuvwxyz'

after = 'cdefghijklmnopqrstuvwxyzab'

개발 환경설정 정리

1. Python 설치

2. IDE Visual Studio Code

3. VSCode 확장프로그램 "Jupyter"

+ 가상환경 `python -m venv "가상환경명"`

+ 파이썬 도구 설치방법 `pip install "도구명"`

ex) `pip install ipykernel`

+ `ipynb(jupyter notebook file)` 생성

Select Kernel → 도구가 있는 환경 지정

문자 자료형 연습문제

<http://ggoreb.com/quiz/운수좋은날.txt>

- "김첨지" 개수 확인 → **49개**

김첨지 글자 사이에 띄어쓰기가 있는 경우도 있음

ex) 김 첨 지 김첨지 김 첨지

List

- N개의 자료를 하나의 자료로 묶어서 사용
- `list = []` 또는 `list = list()` 와 같이 사용
- 문자와 마찬가지로 indexing, slicing 기능 제공
- Dictionary와 함께 매우 자주 사용되는 자료형태

JSON 데이터 활용

1. requests 도구 설치

`pip install requests`

2. JSON 구조를 쉽게 파악

크롬 확장프로그램 `json formatter`

리스트 자료형 연습문제

<http://ggoreb.com/python/json/data4.jsp>

- "q" 데이터 추출하기 (indexing)

영화

API

[https://api.themoviedb.org/3/search/movie?api_key=cba95d401a14ab806ffc13a5052aab89&query=검색어](https://api.themoviedb.org/3/search/movie?api_key=<u>cba95d401a14ab806ffc13a5052aab89</u>&query=검색어)

[https://api.themoviedb.org/3/search/movie?api_key=eb5dea08ca14c5bdd7b9d451b555a0b2&query=검색어](https://api.themoviedb.org/3/search/movie?api_key=<u>eb5dea08ca14c5bdd7b9d451b555a0b2</u>&query=검색어)

(사용량 초과로 실행이 안되면) <http://ggoreb.com/api/movie.jsp>

포스터 이미지 경로

<https://image.tmdb.org/t/p/w185/qYczuua2tgjfxcdtLNDC0n4mOHZ.jpg>

Dictionary 자료형

중괄호 사용 ex) { 'a': 1, 'b': 2, 'c': 3 }

1. 리스트나 문자와 다르게 인덱스가 아닌 Key를 지정해서 조회
- dic['name'] 또는 dic.get('name')
2. 리스트와 문자에서 가능한 슬라이싱 기능 없음
3. Key가 존재하는지 확인하려면 in 사용
4. 딕셔너리 관련 함수 (keys, values, items, clear)

비교 연산자 우선순위 (and, or)

주소 = '부산'

자격증 = True

경력 = 2

자격증이 있거나 경력이 3년 이상이고 주소가 서울인 사람

자격증 == True or 경력 >= 3 and 주소 == '서울'

조건문

○ if - elif - else

1) 콜론 :

2) 들여쓰기

3) 조건식은 항상 True / False

4) 조건에 맞으면 if 아니면 elif

모든 조건에 맞지 아니면 else

→ 아래의 조건은 건너뛴 조건까지 포함

착시 / 착청

소리

<https://www.vocabulary.com/dictionary/laurel>

색상

[https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=tophoonn
&logNo=220285765406](https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=tophoonn&logNo=220285765406)

움직임

[https://postfiles.pstatic.net/20090711_172/panem_1247299241898qVE6y_gif/c1c
2b3fabfecb3fac5d7bdbac6ae_gbmaster_panem.gif?type=w2](https://postfiles.pstatic.net/20090711_172/panem_1247299241898qVE6y_gif/c1c2b3fabfecb3fac5d7bdbac6ae_gbmaster_panem.gif?type=w2)

조건문 연습문제) 평년 / 윤년

윤년인지 평년인지 구분하기

조건 1) 4로 나누어 떨어지는 해는 “윤년” (2004, 2016, ...)

조건 2) 100으로 나누어 떨어지는 해는 “평년” (1900, 2100, ...)

조건 3) 400으로 나누어 떨어지는 해는 “윤년” (1600, 2000, ...)

조건 4) 그 외 모든 해는 “평년” (1997, 2003, 2009, ...)

연습) 알람 시계 일찍 설정하기

조건 1) 24시간 표현으로 하루의 시작은 0:0 이고 끝은 23:59

조건 2) 시간을 입력할 때 시와 분을 띄어쓰기로 구분하고 불필요한 0은 사용하지 않음

ex) 10:10 → 10 10 / 00:10 → 0 10

조건 3) 입력된 시간에서 45분 앞당긴 결과 출력

ex) 14 45 → 14 00 / 23 40 → 22 55 / 0 45 → 0 0 / 0 30 → 23 45

```
value = input().strip().split()
```

```
h = int(value[0])
```

```
m = int(value[1])
```

반복문

○ while

- 1) 콜론 :
- 2) 들여쓰기
- 3) 조건식은 항상 True / False
- 4) 무한히 반복되지 않도록 조건식 주의

ex) while count < 5:

count += 1 # 반복할 때 마다 계속 증가시켜서 언젠가는 조건에 맞지
않도록

조건문 연습문제) 야구게임

숫자 3개를 입력해서 상황에 따라 스트라이크, 볼, 아웃을 판단하기

문제 설명) <http://ggoreb.com/quiz/baseball.html>

```
import requests as req
```

```
res = req.get('http://ggoreb.com/quiz/baseball.txt')
```

```
text = res.text
```

```
정답 = [ text[0], text[1], text[2] ]
```

```
입력 = [ '2', '1', '6' ]
```

7.22.(수)

조건문 연습문제) 야구게임 - 하드코딩, 답 직접 입력

```
import requests as req
res = req.get('http://ggoreb.com/quiz/baseball.txt')
text = res.text
정답 = [ text[0], text[1], text[2] ]
입력 = [ '1', '6', '2' ]
strike = 0 # 초기화
ball = 0
out = 0

if 정답[0] == 입력[0]: strike += 1
elif 정답[0] in 입력: ball += 1
else: out += 1

if 정답[1] == 입력[1]: strike += 1
elif 정답[1] in 입력: ball += 1
else: out += 1

if 정답[2] == 입력[2]: strike += 1
elif 정답[2] in 입력: ball += 1
else: out += 1

print(f'{strike}S {ball}B')
```

조건문 연습문제) 야구게임 - 반복문 적용

```
import requests as req
res = req.get('http://ggoreb.com/quiz/baseball.txt')
text = res.text
정답 = [ text[0], text[1], text[2] ]
입력 = [ '1', '6', '2' ]
strike = 0 # 초기화
ball = 0
out = 0
i = 0
while i <= 3:
    if 정답[i] == 입력[i]: strike += 1
    elif 정답[i] in 입력: ball += 1
    else:
        out += 1

print(f'{strike}S {ball}B')
```

조건문 연습문제) 야구게임 - 사용자 입력 + 답 입력 반복

```
import requests as req
res = req.get('http://ggoreb.com/quiz/baseball.txt')
text = res.text
정답 = [ text[0], text[1], text[2] ]
# 입력 = [ '1', '6', '2' ]
con = True
while con:
    입력 = list(input())
    strike = 0 # 초기화
    ball = 0
    out = 0
    i = 0
    while i < 3:
        if 정답[i] == 입력[i]: strike += 1
        elif 정답[i] in 입력: ball += 1
        else: out += 1
        i += 1
    if strike == 3:
        con = False
    print(f'{strike}S {ball}B')
```

가위바위보 게임 규칙 확인

```
com = '가위'  
player = '바위'
```

```
if com == player:  
    print('비겼습니다.')  
elif com == '가위':  
    if player == '바위':  
        print('플레이어 승리!')  
    else:  
        print('컴퓨터 승리!')  
elif com == '바위':  
    if player == '보':  
        print('플레이어 승리!')  
    else:  
        print('컴퓨터 승리!')
```

가위바위보 게임 규칙 확인

가위	바위	보	
0	1	2	
나 + 1 == 상대방 <=== 내가 졌음			
보 + 1 == 3			
(보 + 1) % 3 == 상대방			

가위바위보 게임 규칙 확인

가위0 바위1 보2

```
import random
```

```
상대방 = random.randint(0, 2)
```

```
나 = int(input())
```

```
if (나 + 1) % 3 == 상대방:
```

```
    print('상대방 승!')
```

```
elif 나 == 상대방:
```

```
    print('비김')
```

```
else:
```

```
    print('나 승!')
```


반복문 연습문제) 숫자 개수 확인하기

1 ~ 100 사이의 숫자 중 3을 포함하고 있는 숫자와 그 개수를 출력하기

3
13
23
30
31
32
33
34
35

37
38
39
43
53
63
73
83
93

총 개수:
19개

반복문

○ for

1) 콜론 :

2) 들여쓰기

3) 리스트

4) 반복횟수가 정해져 있음

로또 번호 생성하기 (6개, 중복될 수 있음)

요구사항) 중복된 번호가 들어가지 않도록 코드 수정하기

```
import random
```

```
로또번호 = list() # []
```

```
n = 0
```

```
while n < 6:
```

```
    n += 1
```

```
    로또번호.append( random.randint(1, 45) )
```

```
로또번호
```

로또 번호 생성하기

```
import random
로또번호 = list() # []
while len(로또번호) < 6:
    temp = random.randint(1, 45)
    if temp not in 로또번호:
        로또번호.append( temp )
로또번호
```

로또 번호 생성하기

```
import random
로또번호 = set()
while len(로또번호) < 6:
    temp = random.randint(1, 45)
    로또번호.add( temp )
로또번호
```

소나기 칩 반복 출력하기

```
import requests
```

```
url = 'http://ggoreb.com/python/file/소나기.txt'
```

```
res = requests.get(url)
```

```
res.encoding = 'utf8'
```

```
text = res.text
```

```
칩위치 = text.find('칩')
```

```
마침표위치 = text.find('.', 칩위치 + 1)
```

```
print(text[칩위치:마침표위치 + 1])
```

소나기 침 반복 출력하기 풀이

```
import requests
url = 'http://ggoreb.com/python/file/ 소나기.txt'
res = requests.get(url)
res.encoding = 'utf8'
text = res.text
```

```
마침표위치 = 0
```

```
while True:
    침위치 = text.find('침', 마침표위치)
    if 침위치 == -1: break
    마침표위치 = text.find('.', 침위치 + 1)
    print(text[침위치:마침표위치 + 1])
```

각 자리 숫자 합 출력하기

```
num = 215179  
total = 0
```

```
n1 = num % 10  
total += n1
```

```
n2 = (num // 10) % 10  
total += n2
```

```
n3 = (num // 100) % 10  
total += n3
```

```
n4 = (num // 1000) % 10  
total += n4
```

```
n5 = (num // 10000) % 10  
total += n5
```

```
n6 = (num // 100000) % 10  
total += n6
```

```
print(total)
```


각 자리 숫자 합 출력하기 풀이

```
num = 215179
```

```
total = 0
```

```
while True:
```

```
    n = num % 10
```

```
    total += n
```

```
    num //= 10
```

```
    if num == 0: break
```

```
print(total)
```

로또 1등 당첨 도전

1. 1등 번호 6자리 준비

2. 도전할 번호 입력

- 랜덤으로 6개 번호 생성

(너무 오래 걸릴수 있으므로 미리 3개는 입력!)

3. 1등 번호와 도전할 번호 비교

로또 1등 당첨 도전

```
import random
```

```
당첨번호 = [6, 11, 16, 19, 21, 32]
```

```
고정번호 = [6, 11, 16]
```

```
로또번호 = set(고정번호)
```

```
while len(로또번호) < 6:
```

```
    로또번호.add(random.randint(1, 45))
```

```
로또번호 = sorted(list(로또번호))
```

```
print(f'나의 로또번호: {로또번호}')
```

```
if 로또번호 == 당첨번호:
```

```
    print('성공!')
```

로또 확률 계산기

<https://parkminkyu.github.io/newLotto/lottoWin.html>

초코칩 단어 개수 세기 연습

1) 띄어쓰기 기준 각 단어를 분리

ex) "안촉촉한 초코칩 ..." → ["안촉촉한", "초코칩", ...]

2) 각 단어와 등장횟수 출력

ex) 안촉촉한 6

초코칩 6

나라에 2

초코칩이 4

...

```
import requests
url = 'http://ggoreb.com/python/file/chocochip.txt'
res = requests.get(url)
res.encoding = 'ms949'
text = res.text
```

초코칩 단어 개수 세기 연습

```
import requests
url = 'http://ggoreb.com/python/file/chocochip.txt'
# 초코칩      초코칩,
res = requests.get(url)
res.encoding = 'ms949'
text = res.text
words = text.split()
등장단어 = {}
for word in words:
    if word in 등장단어:
        등장단어[word] += 1
    else:
        등장단어[word] = 1

print(등장단어)
```

최대 공약수 구하기

$$a = 15, b = 6$$

- 1) a 와 b 중 적은 수 확인
- 2) 적은 수로 큰 수 나누기
- 3) 나머지가 0인 경우 최대 공약수 (반복 종료)
- 4) 현재 적은 수로 나누어지지 않으면 적은 수 - 1
- 5) 1이 될때까지 반복 후 종료

$$\text{최소 공배수} = (a * b) / \text{최대 공약수}$$

최대 공약수 구하기

```
a, b = 15, 6
for n in range(min(a, b), 0, -1):
    # print(n)
    if a % n == 0 and b % n == 0:
        print(n)
        break
```


최대 공약수 구하기 (파이썬의 math 도구 사용)

```
import math
```

```
a, b = 15, 6
```

```
최대공약수 = math.gcd(a, b)
```

```
최소공배수 = math.lcm(a, b)
```

```
print(최대공약수, 최소공배수)
```

리스트 + 딕셔너리 (반복문 적용)

미니 블로그 데이터를 아래와 같은 형식으로 출력하기

제목: 리액트에서 리스트 렌더링하기

댓글수: 5개

실제로 개발하다보면 `map` 함수를 진짜 많이 쓰는 것 같아요 😊

...

```
import requests
```

```
res = requests.get('http://ggoreb.com/api/mini-blog.jsp')
```

```
data = res.json()
```

API

뉴스

<https://newsapi.org/v2/top-headlines?country=kr&apiKey=9f5baf7d9f3f42879a20d7d19d9886e4>

<https://newsapi.org/v2/top-headlines?country=us&apiKey=9f5baf7d9f3f42879a20d7d19d9886e4>

날씨

<https://api.openweathermap.org/data/2.5/weather?lat=35.1728639953646&lon=129.130680529903>

[&units=metric&appid=6edee3c2aa182bc44d18ccb204c98a31](https://api.openweathermap.org/data/2.5/weather?lat=35.1728639953646&lon=129.130680529903&units=metric&appid=6edee3c2aa182bc44d18ccb204c98a31)

유튜브

<https://www.googleapis.com/youtube/v3/search?key=AlzaSyBcYq5zgED0rgC3aX00om0ThqbRPQ5V020&part=snippet&q=월드컵>

<http://ggoreb.com/api/youtube.jsp> [youtube2.jsp](#) [youtube3.jsp](#)

영화

https://api.themoviedb.org/3/search/movie?api_key=cba95d401a14ab806ffc13a5052aab89&query

=서울

연습문제) 거스름돈

당신은 음식점의 계산을 도와주는 점원입니다.

거스름 돈으로 사용할 수 있는 동전은 500원, 100원, 50원, 10원입니다.

손님에게 거슬러 주어야 할 돈이 N원일 때
거슬러 주어야 할 동전의 최소 개수를 구하세요.

n = 1260

count = 0

연습문제) 거스름돈 (여러번 빼기로 해결)

```
n = 1260
```

```
count = 0
```

```
while n > 0:
```

```
    if n >= 500: n -= 500
```

```
    elif n >= 100: n -= 100
```

```
    elif n >= 50: n -= 50
```

```
    elif n >= 10: n -= 10
```

```
    count += 1
```

```
print(count)
```

연습문제) 거스름돈 (나누기로 효율성 높임)

```
n = 1260
count = 0

coin = n // 500 # 동전 개수
count += coin
n %= 500

coin = n // 100 # 동전 개수
count += coin
n %= 100

coin = n // 50 # 동전 개수
count += coin
n %= 50

coin = n // 10 # 동전 개수
count += coin
n %= 10

print(count)
```

연습문제) 거스름돈 (최종 완성)

```
n = 1260
count = 0
for money in [500, 100, 50, 10]:
    coin = n // money
    count += coin
    n %= money
print(count)
```

7.23.(목)

연습문제) 1이 될 때까지

어떠한 수 N 에 1이 될 때까지 아래 두 과정 중 하나의 연산을 수행해야 됩니다.

1) N 에서 1을 뺍니다.

2) N 을 K 로 나눕니다.

단, 두번째 연산은 N 이 K 로 나누어 떨어질 때만 수행 됩니다.

ex) N 이 17, K 가 4 일때

1단계) $17 - 1 \rightarrow 16$ (나누어 떨어지지 않으므로)

2단계) $16 / 4 \rightarrow 4$ (나누어 떨어짐)

3단계) $4 / 4 \rightarrow 1$ (나누어 떨어짐)

1이 될 때까지 수행해야 하는 최소 횟수를 구하세요.

입력 예시) 17 4

출력 예시) 3

연습문제) 1이 될 때까지 (나머지를 구해서 한꺼번에 빼기)

N, K = 32, 11

count = 0

while N > 1:

if N % K == 0:

N //= K

count += 1

else:

뺄값 = N % K

N -= 뺄값

count += 뺄값

K가 N보다 작아지면 나머지 연산자에 의해 0이
되어버림

ex) N = 2 K = 11

N -= 2 → 0

print(f'현재 N: {N}, 횟수: {count}')

연습문제) 1이 될 때까지 (횟수가 1번 더 늘어나는 버그 수정)

N, K = 32, 11

count = 0

while N > 1:

if N % K == 0:

N //= K

count += 1

else:

N이 K보다 작아지면 K로 나눌 수 없으므로, (N - 1)만큼 한 번에 빼고 종료

if N < K:

count += (N - 1)

N = 1

print(f'현재 N: {N}, 횟수: {count}')

break

뺄값 = N % K

N -= 뺄값

count += 뺄값

print(f'현재 N: {N}, 횟수: {count}')

함수

어떠한 값을 입력하면 정의된 절차에 따라 결과물을 내는 것

1. 함수명
2. 반환값
3. 매개변수

함수는 자료형으로 사용 가능

1. 함수는 변수에 저장할 수 있음
2. 함수는 다른 함수의 매개변수로 전달될 수 있음
3. 함수가 실행되려면 소괄호 () 사용

그래프 출력하기

도구 설치 - `pip install matplotlib`

matplotlib의 pyplot 사용

```
import matplotlib.pyplot
```

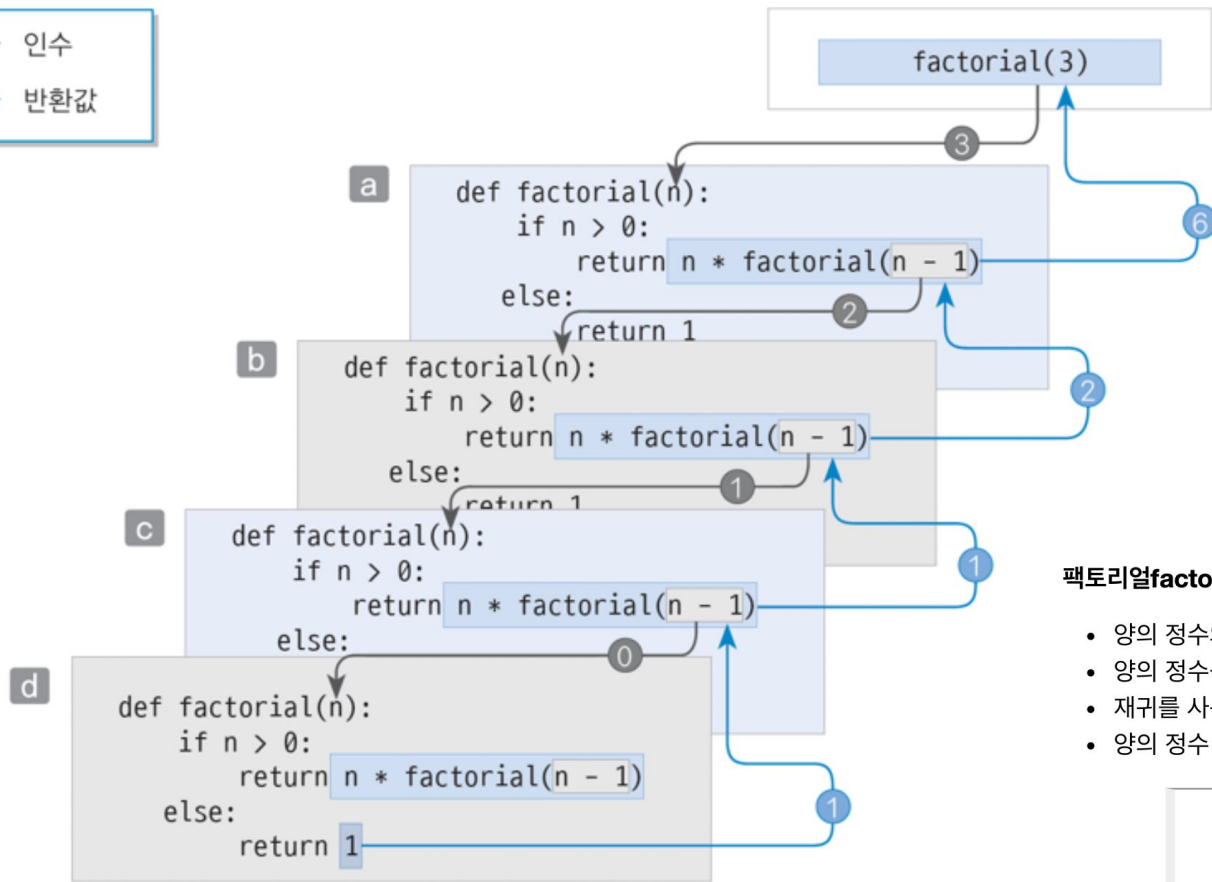
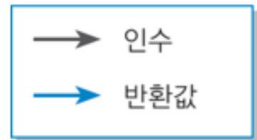
```
matplotlib.pyplot.plot([10, 1, 3, 5, 4], marker='*', color='red')
```

함수 문제 (1부터 n까지 합 구하기)

$$a_n = \sum_{k=1}^n \frac{k(k+1)}{2}$$

$$a_n = \frac{n(n+1)(n+2)}{6}$$

재귀 호출 recursive call



팩토리얼 factorial

- 양의 정수의 곱을 구하는 문제
- 양의 정수를 순서대로 곱한다는 의미로 순차 곱셈이라고도 함
- 재귀를 사용하는 대표적인 예
- 양의 정수 n 의 팩토리얼($n!$)의 재귀적 정의

- $0! = 1$
- $n > 0$ 이면 $n! = n \times (n - 1)!$

최대 공약수

GCD : Greatest Common Divisor

A, B 두 수 중 작은 수를 기준으로 1씩 감소시키면서

A와 B가 나누어 떨어지는 숫자

유클리드 호제법Euclidean algorithm

- x와 y가 있고 x를 y로 나눈 나머지 r이 있을 때
- x와 y의 최대 공약수는 y와 r의 최대 공약수와 같다

단계	A	B	R (나머지)
1	192	162	30
2	162	30	12
3	30	12	6
4	12	6	0

최대 공약수

최소 공배수

LCM : Least Common Multiple

A, B 두 수 중 큰 수를 기준으로 1씩 증가시키면서

A와 B가 나누어 떨어지는 숫자

유클리드 호제법 **Euclidean algorithm**

- x 와 y 가 있고 $x * y$ 를 최대 공약수로 나눈 몫은 최소 공배수이다

변수의 유효범위

○ 함수 내에서 선언된 변수는 지역변수

def 함수명(매개변수):

age = 10

 return 반환값

※ if, while, for 내에서 선언된 변수는 전역변수

Lambda

함수명 → 변수명

매개변수 → **lambda** 매개변수

반환값 → : 반환값

ex) minus = lambda a, b: a - b

람다 연습문제

```
def myfunc(numbers):  
    result = []  
    for number in numbers:  
        if number > 5:  
            result.append(number)  
    return result
```

```
myfunc2 = lambda numbers: [ n for n in numbers if n > 5 ]
```

```
result = myfunc2([2, 3, 4, 5, 6, 7, 8])
```

```
print(result)
```

코드를 간결하게 만드는 한줄 표현식들

1. if → 조건부 표현식
2. for → 리스트 컴프리헨션
3. 함수 → 람다식

파일 종류

1. 텍스트

메모장, 편집기로 내용을 확인할 수 있는 형태

- 인코딩 종류

utf-8 - 한글 등 특수문자 3Byte

euc-kr (cp949, ms949) - 한글 등 특수문자 2Byte

2. 바이너리

특수한 프로그램을 사용해서 내용을 확인할 수 있는 형태

서울시 기온 그래프 그리기

```
f = open('seoul_기온.csv', 'rt')
date_list = []
min_list = []
for data in f:
    #      0      1      2      3
    # 2021-01-02,-5,-8.4,-1.4
    date_list += [data.split(',')[0]]
    min_list.append(data.split(',')[2])
f.close()

date_list = [ date for date in date_list[1:] ]
min_list = [ float(min) for min in min_list[1:] ]

import matplotlib.pyplot as plt
plt.rcParams['axes.unicode_minus'] = False
plt.plot(min_list)
plt.xticks(range(len(min_list)), date_list, rotation=45)
plt.show()
```


Data Uri (base64 이미지)

```
file = 'c:/image.png'
```

```
import base64
```

```
file = open(file, 'rb')
```

```
base64_string = base64.b64encode(file.read())
```

```
file.close()
```

```
print('' % base64_string.decode())
```

텍스트 파일 내용 읽기

read() - 내용 전체

for 변수 in 파일객체: - 한줄씩

readline() - 한줄만 (\n 문자 기준)

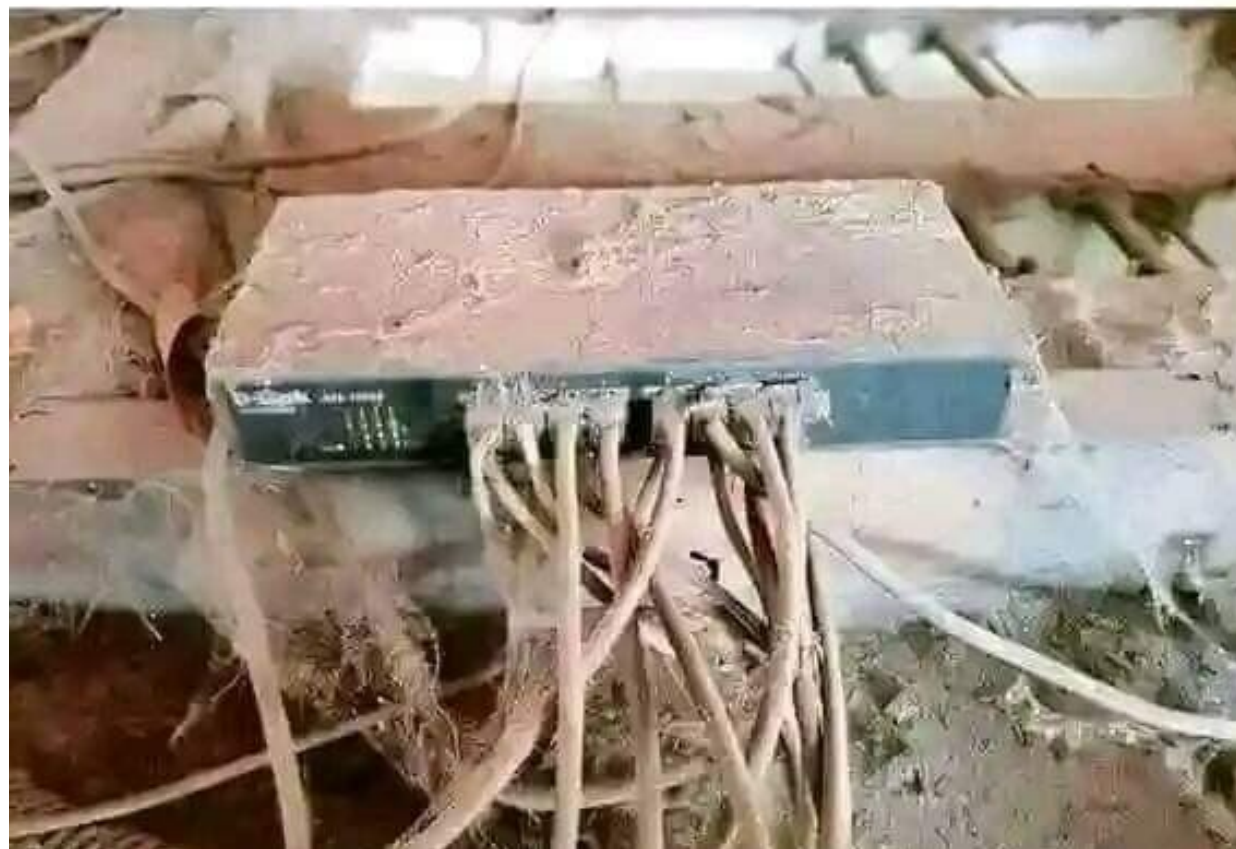
readlines() - 내용 전체 (리스트)

텍스트 파일 내용 쓰기

write() - 문자를 파일의 내용으로 쓰기

writelines() - 리스트를 파일의 내용으로 쓰기

If it works, don't touch it



파일쓰기 연습문제 (시스템 로그 파일 생성)

프로그램이 실행되면서 발생하는 가상의 기록(로그)을 `system.log` 파일에 텍스트 형식으로 누적 저장합니다.

조건 1) 기존 파일 내용을 지우지 않고 이어서 작성

조건 2) 각 로그는 줄바꿈하며 한 줄에 하나씩 작성

```
import datetime
```

```
# 현재 날짜와 시간을 구하기
```

```
current_time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
# 기록할 가짜 로그 데이터
```

```
log_messages = [
```

```
    "사용자가 관리자 페이지에 로그인했습니다.",
```

```
    "데이터베이스 연결에 성공했습니다.",
```

```
    "경고: 디스크 용량이 80% 이상 채워졌습니다.",
```

```
    "에러: 파일 업로드 중 예기치 못한 오류가 발생했습니다."
```

```
]
```

```
print("로그 생성을 시작합니다...")
```

```
# [문제] 반복문을 사용하여 system.log 파일에 로그를 한 줄씩 누적해서 쓰기
```

```
print("로그 파일 작성 완료!")
```

파일쓰기 연습문제 (이미지 다운로드)

인터넷 주소(URL)에 있는 이미지 파일을 파이썬 코드로 다운로드하여 내 컴퓨터에 `python-image.jpg` 로 저장합니다.

```
import requests
```

```
# 다운로드할 파이썬 로고 이미지 주소입니다.
```

```
image_url = "http://ggoreb.com/python/images/python-image.jpg"
```

```
print("웹에서 이미지를 요청하는 중...")
```

```
# requests를 사용해 이미지 데이터를 가져옵니다.
```

```
response = requests.get(image_url)
```

```
image_bytes = response.content
```

```
print("이미지 다운로드 완료! 이제 파일로 저장합니다...")
```

```
# [문제] 받아온 이미지 바이너리 데이터(image_bytes)를 'python-image.jpg' 파일로 저장하기
```

```
print("이미지 파일 저장 완료! 폴더에서 이미지 파일을 확인해보세요.")
```

7.25.(금)

해시 Hash

- 해시(Hash)

임의의 길이 데이터를 고정된 길이의 (거의) 고유한 값으로 변경하는 것

- 해시값 / 해시코드 (Hash Code)

일반적으로 해시값 또는 해시코드 라고 부름

- 해시 함수 (Hash Function)

해시를 생성해주는 함수

* **SHA-256**의 경우 **2**의 **256**승을 의미하고

전세계 **80**억 인구가 **1**초에 하나씩 **136**해 년 동안 시도해야 맞출 정도의 확률
결론은 중복될 확률 **0%**

문자열 해시 (비밀번호 암호화)

```
import hashlib
```

```
old_pwd = '1234'
```

```
m = hashlib.sha256()
```

```
m.update(old_pwd.encode())
```

```
new_pwd = m.hexdigest()
```

```
# new_pwd = hashlib.sha256(old_pwd.encode()).hexdigest()
```

```
print(new_pwd)
```

정규 표현식으로 할 수 있는 행위

1. 검사

- 문자열 찾아내기
- `find (search())`

2. 추출

- 문자열 잘라내기
- `slicing (findall())`

3. 치환

- 문자열 변경하기
- `replace (sub())`

정규 표현식 기본 코드

```
import re
```

```
규칙 = re.compile( '[^가-힣]' )
```

```
문장 = '가나라abcd가나라라다'
```

```
결과 = 규칙.search( 문장 )
```

```
if 결과: print('통과')
```

```
else: print('불가')
```

1. re 도구 불러오기
2. pattern 지정 - compile()
3. 확인 - search()
 - 1개 확인: search() N개 확인: findall()

정규 표현식 사용 기호

[] 검사할 문자

{ } 등장 횟수

. 모든 문자

* 등장 횟수 (0번 이상)

+ 등장 횟수 (1번 이상)

^ 시작

\$ 종료

\ 특수문자

() 그룹

? 등장 횟수 (0 ~ 1번, Non-Greedy)

정규 표현식

== 단독 사용이 가능한 기호 ==

[] 대괄호

지정된 문자에 “또는” 개념 적용

범위표현 - (하이픈)

[^] 부정

\d 숫자 **\s** 공백 **\w** 문자

. 모든문자 (줄바꿈 빼고)

단, 대괄호 안의 .은 실제 마침표

정규 표현식

== 단독 사용 불가, 위의 기호와 같이 사용해야 하는 기호==

* 0개 ~ N개

+ 1개 ~ N개

{ } {숫자} 숫자만큼

{숫자1,숫자2} 숫자1~숫자2

? 0개 ~ 1개

greedy스러운 기호를 non-greedy 변경

^ 시작 \$ 종료

| 여러개의 패턴 (or)

() 그룹화

한글, 일본어, 한자 범위 표현

```
pattern = re.compile(r'[가-힣ぁ-んァ-ンㄱ-ㅎ\u4E00-\u9FFF]+')
```

언어/문자 종류	코드 범위 (16진수)	정규식 표현	예시 문자	비고
한글	AC00-D7A3	[가-힣]	가, 나, 다	완성형 한글 (11172자)
히라가나	3040-309F	[ぁ-ん]	あ, い, う	일본어 고유 문자
가타카나	30A0-30FF	[ァ-ン]	ア, イ, ウ	외래어·이름 표기
반각 가타카나	FF65-FF9F	[ㄱ-ㅎ]	ア, ハ, シ	옛 Shift-JIS 호환용
한자	4E00-9FFF	[\u4E00-\u9FFF]	中, 國, 漢	현대 중국어 한자 대부분
한자 확장 A	3400-4DBF	[\u3400-\u4DBF]	ㄱ, ㄴ	드물게 쓰이는 한자
(선택) 확장 B~F	20000-2EBEF	[\u20000-\u2EBEF]	ㄷ 등	고대 문자·희귀 한자 포함

7.27.(월)

로또 번호

<http://ggoreb.com/python/html/number.html>

```
import requests  
res = requests.get('http://ggoreb.com/python/html/number.html')  
res.encoding = None  
text = res.text
```

```
import re  
pattern = re.compile(r'ball[0-9]+>([0-9]+)')  
res = pattern.findall(text)  
print(len(res))  
print(res)
```

```
<span class="ball_645_lrg_ball1">2</span>
```


숨어있는 문자 찾기

```
import requests  
res = requests.get('http://ggoreb.com/quiz/special.html')  
res.encoding = None  
text = res.text  
idx = text.find('%%$')  
text = text[idx:]
```

```
import re  
pattern = re.compile(r'[a-z]+')  
res = pattern.findall(text)  
print("".join(res))  
print(len(res))  
print(res)
```

동일한 패턴 문자 찾기

문서 : <http://ggoreb.com/quiz/pattern.html>

- 조건에 맞는 문자 찾기 (소문자 10개)

```
<!--  
find rare characters in the mess below:  
-->  
  
<!--  
kAewtIoYgcFQaJNhHVGxXDlQmzfcpYbzxIWrvCqsmUbCunkfxZWDZjUZMiGqhRRiUvGmYmynJIHEmbT  
MUKLECKdCthezSYBpIEIRnZugFAxDRtQPpyeCBgBfArVvvguRXLvkAdLOeCKxsDUvBBCwdpMMWmuELeG  
ENihrpCLhujoBqPRDPvfzcadMMmbkmkzCCzoTPfbRlzbQMbImxTxNniNoCufprWXxgHZpIdkoLCrHJq  
vYuyJFCZtqXLhWiYzOXeglkzhVJIWmeUySGuFVmlTCyMshQtvZpPwulbOHNoBauwvuJYCmqznOBgByPw  
T891 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1041 1042 1043 1044 1045 1046 1047 1048 1049 1050 1051 1052 1053 1054 1055 1056 1057 1058 1059 1060 1061 1062 1063 1064 1065 1066 1067 1068 1069 1070 1071 1072 1073 1074 1075 1076 1077 1078 1079 1080 1081 1082 1083 1084 1085 1086 1087 1088 1089 1090 1091 1092 1093 1094 1095 1096 1097 1098 1099 1100 1101 1102 1103 1104 1105 1106 1107 1108 1109 1110 1111 1112 1113 1114 1115 1116 1117 1118 1119 1120 1121 1122 1123 1124 1125 1126 1127 1128 1129 1130 1131 1132 1133 1134 1135 1136 1137 1138 1139 1140 1141 1142 1143 1144 1145 1146 1147 1148 1149 1150 1151 1152 1153 1154 1155 1156 1157 1158 1159 1160 1161 1162 1163 1164 1165 1166 1167 1168 1169 1170 1171 1172 1173 1174 1175 1176 1177 1178 1179 1180 1181 1182 1183 1184 1185 1186 1187 1188 1189 1190 1191 1192 1193 1194 1195 1196 1197 1198 1199 1200 1201 1202 1203 1204 1205 1206 1207 1208 1209 1210 1211 1212 1213 1214 1215 1216 1217 1218 1219 1220 1221 1222 1223 1224 1225 1226 1227 1228 1229 1230 1231 1232 1233 1234 1235 1236 1237 1238 1239 1240 1241 1242 1243 1244 1245 1246 1247 1248 1249 1250 1251 1252 1253 1254 1255 1256 1257 1258 1259 1260 1261 1262 1263 1264 1265 1266 1267 1268 1269 1270 1271 1272 1273 1274 1275 1276 1277 1278 1279 1280 1281 1282 1283 1284 1285 1286 1287 1288 1289 1290 1291 1292 1293 1294 1295 1296 1297 1298 1299 1300 1301 1302 1303 1304 1305 1306 1307 1308 1309 1310 1311 1312 1313 1314 1315 1316 1317 1318 1319 1320 1321 1322 1323 1324 1325 1326 1327 1328 1329 1330 1331 1332 1333 1334 1335 1336 1337 1338 1339 1340 1341 1342 1343 1344 1345 1346 1347 1348 1349 1350 1351 1352 1353 1354 1355 1356 1357 1358 1359 1360 1361 1362 1363 1364 1365 1366 1367 1368 1369 1370 1371 1372 1373 1374 1375 1376 1377 1378 1379 1380 1381 1382 1383 1384 1385 1386 1387 1388 1389 1390 1391 1392 1393 1394 1395 1396 1397 1398 1399 1400 1401 1402 1403 1404 1405 1406 1407 1408 1409 1410 1411 1412 1413 1414 1415 1416 1417 1418 1419 1420 1421 1422 1423 1424 1425 1426 1427 1428 1429 1430 1431 1432 1433 1434 1435 1436 1437 1438 1439 1440 1441 1442 1443 1444 1445 1446 1447 1448 1449 1450 1451 1452 1453 1454 1455 1456 1457 1458 1459 1460 1461 1462 1463 1464 1465 1466 1467 1468 1469 1470 1471 1472 1473 1474 1475 1476 1477 1478 1479 1480 1481 1482 1483 1484 1485 1486 1487 1488 1489 1490 1491 1492 1493 1494 1495 1496 1497 1498 1499 1500 1501 1502 1503 1504 1505 1506 1507 1508 1509 1510 1511 1512 1513 1514 1515 1516 1517 1518 1519 1520 1521 1522 1523 1524 1525 1526 1527 1528 1529 1530 1531 1532 1533 1534 1535 1536 1537 1538 1539 1540 1541 1542 1543 1544 1545 1546 1547 1548 1549 1550 1551 1552 1553 1554 1555 1556 1557 1558 1559 1560 1561 1562 1563 1564 1565 1566 1567 1568 1569 1570 1571 1572 1573 1574 1575 1576 1577 1578 1579 1580 1581 1582 1583 1584 1585 1586 1587 1588 1589 1590 1591 1592 1593 1594 1595 1596 1597 1598 1599 1600 1601 1602 1603 1604 1605 1606 1607 1608 1609 1610 1611 1612 1613 1614 1615 1616 1617 1618 1619 1620 1621 1622 1623 1624 1625 1626 1627 1628 1629 1630 1631 1632 1633 1634 1635 1636 1637 1638 1639 1640 1641 1642 1643 1644 1645 1646 1647 1648 1649 1650 1651 1652 1653 1654 1655 1656 1657 1658 1659 1660 1661 1662 1663 1664 1665 1666 1667 1668 1669 1670 1671 1672 1673 1674 1675 1676 1677 1678 1679 1680 1681 1682 1683 1684 1685 1686 1687 1688 1689 1690 1691 1692 1693 1694 1695 1696 1697 1698 1699 1700 1701 1702 1703 1704 1705 1706 1707 1708 1709 1710 1711 1712 1713 1714 1715 1716 1717 1718 1719 1720 1721 1722 1723 1724 1725 1726 1727 1728 1729 1730 1731 1732 1733 1734 1735 1736 1737 1738 1739 1740 1741 1742 1743 1744 1745 1746 1747 1748 1749 1750 1751 1752 1753 1754 1755 1756 1757 1758 1759 1760 1761 1762 1763 1764 1765 1766 1767 1768 1769 1770 1771 1772 1773 1774 1775 1776 1777 1778 1779 1780 1781 1782 1783 1784 1785 1786 1787 1788 1789 1790 1791 1792 1793 1794 1795 1796 1797 1798 1799 1800 1801 1802 1803 1804 1805 1806 1807 1808 1809 1810 1811 1812 1813 1814 1815 1816 1817 1818 1819 1820 1821 1822 1823 1824 1825 1826 1827 1828 1829 1830 1831 1832 1833 1834 1835 1836 1837 1838 1839 1840 1841 1842 1843 1844 1845 1846 1847 1848 1849 1850 1851 1852 1853 1854 1855 1856 1857 1858 1859 1860 1861 1862 1863 1864 1865 1866 1867 1868 1869 1870 1871 1872 1873 1874 1875 1876 1877 1878 1879 1880 1881 1882 1883 1884 1885 1886 1887 1888 1889 1890 1891 1892 1893 1894 1895 1896 1897 1898 1899 1900 1901 1902 1903 1904 1905 1906 1907 1908 1909 1910 1911 1912 1913 1914 1915 1916 1917 1918 1919 1920 1921 1922 1923 1924 1925 1926 1927 1928 1929 1930 1931 1932 1933 1934 1935 1936 1937 1938 1939 1940 1941 1942 1943 1944 1945 1946 1947 1948 1949 1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033 2034 2035 2036 2037 2038 2039 2040 2041 2042 2043 2044 2045 2046 2047 2048 2049 2050 2051 2052 2053 2054 2055 2056 2057 2058 2059 2060 2061 2062 2063 2064 2065 2066 2067 2068 2069 2070 2071 2072 2073 2074 2075 2076 2077 2078 2079 2080 2081 2082 2083 2084 2085 2086 2087 2088 2089 2090 2091 2092 2093 2094 2095 2096 2097 2098 2099 2100 2101 2102 2103 2104 2105 2106 2107 2108 2109 2110 2111 2112 2113 2114 2115 2116 2117 2118 2119 2120 2121 2122 2123 2124 2125 2126 2127 2128 2129 2130 2131 2132 2133 2134 2135 2136 2137 2138 2139 2140 2141 2142 2143 2144 2145 2146 2147 2148 2149 2150 2151 2152 2153 2154 2155 2156 2157 2158 2159 2160 2161 2162 2163 2164 2165 2166 2167 2168 2169 2170 2171 2172 2173 2174 2175 2176 2177 2178 2179 2180 2181 2182 2183 2184 2185 2186 2187 2188 2189 2190 2191 2192 2193 2194 2195 2196 2197 2198 2199 2200 2201 2202 2203 2204 2205 2206 2207 2208 2209 2210 2211 2212 2213 2214 2215 2216 2217 2218 2219 2220 2221 2222 2223 2224 2225 2226 2227 2228 2229 2230 2231 2232 2233 2234 2235 2236 2237 2238 2239 2240 2241 2242 2243 2244 2245 2246 2247 2248 2249 2250 2251 2252 2253 2254 2255 2256 2257 2258 2259 2260 2261 2262 2263 2264 2265 2266 2267 2268 2269 2270 2271 2272 2273 2274 2275 2276 2277 2278 2279 2280 2281 2282 2283 2284 2285 2286 2287 2288 2289 2290 2291 2292 2293 2294 2295 2296 2297 2298 2299 2300 2301 2302 2303 2304 2305 2306 2307 2308 2309 2310 2311 2312 2313 2314 2315 2316 2317 2318 2319 2320 2321 2322 2323 2324 2325 2326 2327 2328 2329 2330 2331 2332 2333 2334 2335 2336 2337 2338 2339 2340 2341 2342 2343 2344 2345 2346 2347 2348 2349 2350 2351 2352 2353 2354 2355 2356 2357 2358 2359 2360 2361 2362 2363 2364 2365 2366 2367 2368 2369 2370 2371 2372 2373 2374 2375 2376 2377 2378 2379 2380 2381 2382 2383 2384 2385 2386 2387 2388 2389 2390 2391 2392 2393 2394 2395 2396 2397 2398 2399 2400 2401 2402 2403 2404 2405 2406 2407 2408 2409 2410 2411 2412 2413 2414 2415 2416 2417 2418 2419 2420 2421 2422 2423 2424 2425 2426 2427 2428 2429 2430 2431 2432 2433 2434 2435 2436 2437 2438 2439 2440 2441 2442 2443 2444 2445 2446 2447 2448 2449 2450 2451 2452 2453 2454 2455 2456 2457 2458 2459 2460 2461 2462 2463 2464 2465 2466 2467 2468 2469 2470 2471 2472 2473 2474 2475 2476 2477 2478 2479 2480 2481 2482 2483 2484 2485 2486 2487 2488 2489 2490 2491 2492 2493 2494 2495 2496 2497 2498 2499 2500 2501 2502 2503 2504 2505 2506 2507 2508 2509 2510 2511 2512 2513 2514 2515 2516 2517 2518 2519 2520 2521 2522 2523 2524 2525 2526 2527 2528 2529 2530 2531 2532 2533 2534 2535 2536 2537 2538 2539 2540 2541 2542 2543 2544 2545 2546 2547 2548 2549 2550 2551 2552 2553 2554 2555 2556 2557 2558 2559 2560 2561 2562 2563 2564 2565 2566 2567 2568 2569 2570 2571 2572 2573 2574 2575 2576 2577 2578 2579 2580 2581 2582 2583 2584 2585 2586 2587 2588 2589 2590 2591 2592 2593 2594 2595 2596 2597 2598 2599 2600 2601 2602 2603 2604 2605 2606 2607 2608 2609 2610 261
```

정규표현식에서 자주 사용되는 플래그 옵션

플래그	축약 형	주요 역할	추천 활용 상황
re.IGNORECASE	re.I	대소문자 구분 없음	영문 텍스트 검색, 회원가입 ID/이메일 검증
re.MULTILINE	re.M	다중 행 모드 (^, \$가 줄바꿈 기준으로 작동)	여러 줄로 구성된 텍스트의 각 줄 첫/끝 단어 찾기
re.DOTALL	re.S	. 이 줄바꿈(\n)도 포함	HTML/JSON 태그 내 여러 줄 텍스트 긁어오기
re.ASCII	re.A	\w, \b 등을 ASCII 범위(영문/숫자)로 제한	한글/유니코드 제외하고 순수 영문·숫자만 뽑기

매장 데이터

<http://ggoreb.com/python/file/shop.txt>

○ 데이터 구성 형식

- 1) 매장명: 숫자 2자리 / 영어, 숫자, 한글, 띄어쓰기
- 2) 매장설명
- 3) 메뉴 및 가격
- 4) 주소 / 전화번호 / 영업시간 / 휴무일

○ 전체 매장수: 564개

매장 데이터

문제 1) 모든 매장명 추출하기

각 매장 이름이 적힌 줄(예: 01 무시로, 02 삼형제오리)만 모두 추출하세요.

총 매장 수는 **564개** 입니다.

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

- 1) 매장명 줄은 항상 두 자리 숫자와 공백으로 시작합니다.
- 2) 매장명에 한글, 영어, 숫자, 띄어쓰기가 섞여있을 수 있습니다.

매장 데이터

문제 2) 지역(구/군) 이름 추출하기

꺾쇠괄호로 표시된 지역(예: <영도구> <서구>)을 모두 추출하세요.

총 지역 수는 **15개** 입니다.

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result)
```

[참고사항]

1) <영도구>처럼 괄호 다음 바로 문자가 나오는
경우와

<서구>처럼 공백이 끼여있는 경우가 있습니다.

매장 데이터

문제 3) 주소 추출하기

정보 줄 (**부산시 ...** / 전화번호 / 영업시간 / 휴무일) 에서 **주소 부분만** 잘라내세요.
(전화번호, 영업시간, 휴무일은 제외하고 **"부산시"부터 첫 번째 / 전까지만**)

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

1) .+ 표현은 greedy 해서 마지막 / 까지 다 가져옵니다.

2) ? 를 붙여서 .+? 처럼 쓰면 non-greedy로 바뀌어서

첫번째로 등장하는 / 에서 멈출 수 있습니다.

매장 데이터

문제 4) 전화번호 추출하기

정보 줄 (부산시 ... / **전화번호** / 영업시간 / 휴무일) 에서 **전화번호 부분만** 잘라내세요.
(전화번호에 대한 내용이 빠진 매장이 6개 있습니다)

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

- 1) .+ 표현은 **greedy** 해서 마지막 / 까지 다 가져옵니다.
- 2) ? 를 붙여서 .+? 처럼 쓰면 **non-greedy**로 바뀌어서
첫번째로 등장하는 / 에서 멈출 수 있습니다.
- 3) "전화불가", "전화문의 불가", "1688-1688" 과 같이
전화번호의 형식이 다른 매장도 있습니다.

매장 데이터

문제 5) 메뉴 추출하기

각 매장의 **모든 메뉴**를 추출하세요.

```
import re, requests
res = requests.get('http://ggoreb.com/python/file/shop.txt')
res.encoding = 'utf8'
text = res.text
p = re.compile(r'코드 작성', re.M)
result = p.findall(text)
print(len(result))
print(result[:3])
```

[참고사항]

- 1) | (파이프) 기호는 정규표현식에서
특수 기호 (or, 여러 패턴 중 하나)로 쓰이기 때문에
문자 그대로의 | 를 찾으려면
이스케이프 문자 \ 와 함께 \|로 써야 합니다.
- 2) .+ 는 **greedy** 이기 때문에
한 줄에 | 가 여러 번 나오면 마지막 | 까지
욕심내서
다 가져가 버립니다.

종합 연습문제 (1장 ~ 6장)

문제 1) 회식비 정산하기 (수학/계산 내장함수)

팀 회식을 마치고 총 결제금액을 인원수대로 나누려고 합니다.

1인당 내야 하는 금액과 나머지 금액 구하세요.

또한 100원 단위로만 정산이 가능한 경우를 대비해 반올림 처리를 해주세요.

```
total_price = 187500
```

```
people = 7
```

```
# 코드 작성 (divmod 사용)
```

```
print(f'1인당 {share}원, 나머지 {remain}원은 총무가 부담')
```

```
# 내야 하는 반올림해서 보정한 1인당 금액 (rounded)
```

```
# 코드 작성 (round 사용)
```

```
print(f'반올림 정산 시 1인당 {rounded}원')
```

[참고사항]

1) `divmod()`를 사용하면 몫과 나머지를 한 번에 구할 수 있습니다.

2) `round()`의 두 번째 인자로 반올림 자릿수를 지정할 수 있습니다.

[실행결과]

1인당 26785원, 나머지 5원은 총무가 부담
반올림 정산 시 1인당 26800.0원

종합 연습문제 (1장 ~ 6장)

문제 2) 이번 달 매출 순위표 만들기 (순서/반복 내장함수)
영업팀 직원별 이번 달 매출 데이터가 있습니다.
매출이 높은 순서대로 순위를 매겨 출력하세요.

```
names = ['김민준', '이서연', '박도윤', '최하은', '정지호']  
sales = [3200000, 4550000, 2980000, 4550000, 3870000]
```

```
# 이름과 매출을 하나로 묶고, 매출 내림차순으로 정렬한 뒤  
# 순위(1위부터)와 함께 출력하세요  
# 코드 작성 (zip, sorted, enumerate 사용)
```

[참고사항]

1) sorted()의 key와 reverse 옵션을 활용해보세요.

사용 예1) 딕셔너리 정렬

```
items1 = [{'name': 'a'}, {'name': 'c'}, {'name': 'b'}]  
sorted_items1 = \n    sorted(items1, key=lambda x: x['name'],  
           reverse=True)  
print(sorted_items1)
```

사용 예2) 튜플 정렬

```
items2 = [('김', 20), ('이', 30), ('박', 25)]  
sorted_items2 = sorted(items2, key=lambda x: x[0])  
print(sorted_items2)
```

[실행결과]

0위 이서연 4550000원

1위 최하은 4550000원

...

종합 연습문제 (1장 ~ 6장)

문제 3) 회원가입 폼 유효성 검사하기 (논리/검사 내장함수)

회원가입 시 입력받은 정보가 조건을 만족하는지 검사하는 함수를 만드세요.

```
def check_signup(user_id, password, age):  
    # 조건 1) user_id는 문자열이어야 함  
    # 조건 2) password 길이는 8자 이상이어야 함  
    # 조건 3) age는 정수형이며 14세 이상이어야 함  
    # 세 조건을 모두 만족해야 True, 하나라도 어기면 False  
    # 코드 작성 (isinstance, all 사용)  
    pass
```

```
print(check_signup('minjun01', 'pw12345678', 20)) # True  
print(check_signup('minjun01', 'pw123', 20))        # False (비밀번호 짧음)  
print(check_signup('minjun01', 'pw12345678', '20')) # False (age가 문자열)
```

[참고사항]

- 1) isinstance(값, 타입)으로 자료형을 검사할 수 있습니다.
- 2) 여러 조건을 리스트로 만들고 all()로 한 번에 검사해보세요.

[실행결과]

True
False
False

종합 연습문제 (1장 ~ 6장)

문제 4) 이벤트 쿠폰 코드 만들기 (문자/코드 내장함수)

이벤트 당첨자에게 발급할 쿠폰 코드를 만들려고 합니다.

회원 아이디의 각 글자를 유니코드 숫자로 바꾼 뒤

그 합을 16진수로 표현해서 쿠폰 코드 뒷자리로 사용합니다.

```
user_id = 'minjun01'
```

```
# 1) 각 글자를 유니코드 코드값으로 변환해서 리스트로 만들기
```

```
# 코드 작성 (ord 사용)
```

```
# 2) 코드값의 합을 구하고, 16진수 문자열로 변환하기
```

```
# 코드 작성 (sum, hex 사용)
```

```
print(f'쿠폰코드: EVENT-{hex_code}')
```

[참고사항]

1) ord()는 문자 하나를 정수(유니코드 코드값)로 바꿔줍니다.

2) hex()의 결과에는 앞에 0x가 붙으니 슬라이싱으로 제거하세요.

[실행결과]

쿠폰코드: EVENT-2f2

종합 연습문제 (1장 ~ 6장)

문제 5) 재고 데이터 일괄 처리하기 (기타/응용 내장함수)

창고 재고 시스템에서 받아온

문자열 형태의 재고 수량 데이터를 정리하려고 합니다.

```
stock_raw = ['120', '35', '8', '250', '64']
```

1) 문자열 리스트를 정수 리스트로 변환한 뒤

재고가 50개 미만인 것만 추려내기

코드 작성 (map, filter, lambda 사용)

```
print('재고부족 목록:', low_stock)
```

2) 재고가 100개 이상인 상품은

창고 정리를 위해 절반만 남기고 나머지는

타 지점으로 이관을 위해 리스트를 만드세요. 120 → 60, 250 → 125

코드 작성 (map, lambda 사용)

```
print('이관 수량:', transfer_list)
```

[참고사항]

1) map(함수, 리스트)는 리스트의 모든 요소에 함수를 적용합니다.

2) filter(함수, 리스트)는 조건을 만족하는 요소만 남깁니다.

3) 소수점이 나오지 않도록 정수 나눗셈(//)을 사용하세요.

[실행결과]

재고부족 목록: [35, 8]

이관 수량: [60, 125]

종합 연습문제 (1장 ~ 6장)

문제 6) 거래처 API 응답 데이터 파싱하기 (json)

거래처 시스템에서 아래와 같은 JSON 형태의 주문 데이터를 받았습니다.
배송 대기 중인 주문의 고객명만 추려내세요.

```
import json
response = '''
[
  {"order_id": 101, "customer": "김민준", "status": "배송중"},
  {"order_id": 102, "customer": "이서연", "status": "배송대기"},
  {"order_id": 103, "customer": "박도윤", "status": "배송대기"},
  {"order_id": 104, "customer": "최하은", "status": "배송완료"}
]
```

1) JSON 문자열을 파이썬 객체로 변환하세요

코드 작성

2) status가 '배송대기'인 주문의 customer만 리스트로 뽑으세요

코드 작성

print(waiting_customers)

[참고사항]

1) json.loads()는 JSON 문자열을
리스트/딕셔너리로 변환해줍니다.

[실행결과]

['이서연', '박도윤']

종합 연습문제 (1장 ~ 6장)

문제 7) 경품 이벤트 당첨자 추천하기 (random)

인스타그램 팔로우 이벤트 참여자 명단 중에서 랜덤으로 당첨자를 뽑는 프로그램을 만드세요.

```
import random
```

```
participants = [  
    'user_a', 'user_b', 'user_c', 'user_d', 'user_e',  
    'user_f', 'user_g', 'user_h', 'user_i', 'user_j'  
]
```

1) 참여자 명단을 무작위로 섞은 후 출력하세요
코드 작성 (random.shuffle 사용)

2) 1등 1명, 2등 2명을 중복 없이 뽑으세요
코드 작성 (random.sample 사용)

```
print('1등:', first_prize)  
print('2등:', second_prize)
```

[참고사항]

1) random.shuffle()은

리스트 자체를 무작위로 섞습니다 (반환값 없음)

2) random.sample(리스트, n)은

중복 없이 n개를 뽑습니다.

[실행결과]

참여자 명단: ['user_j', 'user_g', ... 'user_b']

1등: user_g

2등: ['user_d', 'user_c']

종합 연습문제 (1장 ~ 6장)

문제 8) 서버 백업 파일명 자동 생성 & 실행시간 측정 (time, datetime)

서버 백업을 실행할 때마다 현재 시각이 담긴 파일명을 자동으로 만들고 백업 작업(예: 파일 압축)에 걸린 시간을 측정하려고 합니다.

```
import time
import datetime

# 1) 'backup_20260726_153045.zip' 같이
#   현재 일시를 이용해 파일명을 만드세요.
# 코드 작성 (datetime.datetime.now(), strftime 사용)
print(backup_filename)

# 2) 아래 작업에 소요된 시간을 측정하세요
start = time.time()
total = 0
for i in range(30_000_000):
    total += i
# 코드 작성 (걸린 시간 = 현재시각 - start)
print(f'백업 소요시간: {elapsed:.2f}초')
```

[참고사항]

- 1) strftime('%Y%m%d_%H%M%S') 형식으로 문자열을 만들 수 있습니다.
- 2) time.time()은 실행 시점의 유닉스 타임스탬프를 반환합니다.

[실행결과]

backup_20260726_105920.zip
백업 소요시간: 1.91초

이메일 발송 코드 - [email.ipynb](https://github.com/Amugona/email.ipynb)

```
import shutil
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
```

Gmail 계정 정보 (앱 비밀번호 사용!)

```
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
SENDER = "amugona.2606@gmail.com"
PASSWORD = "orzkgmnwuhmubgqp"
RECIPIENT = "seorab@naver.com"
```

여러명에게 발송할 경우 리스트로 지정

```
# RECIPIENT = ["seorab@naver.com", "seorab81@gmail.com"]
```

1. 디스크 사용량 확인

```
def check_disk_usage(path="/", threshold=80):
    total, used, free = shutil.disk_usage(path)
    usage = used / total * 100
    return usage, threshold
```

2. 이메일 발송 함수

```
def send_email(subject, body):
    msg = MIMEMultipart()
    msg["From"] = SENDER
    msg["To"] = RECIPIENT
```

```
# 여러명에게 발송할 경우 구분자를 넣어 문자열로 변환
# msg["To"] = ', '.join(RECIPIENT)
```

```
msg["Subject"] = subject
msg.attach(MIMEText(body, "html"))
```

try:

```
with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
    server.starttls() # TLS 암호화 연결
    server.login(SENDER, PASSWORD) # 로그인
    server.send_message(msg) # 메일 발송
```

except Exception as e:

```
print(f"❌ 발송 실패 오류 발생: {e}")
```

3. 메인 로직

```
if __name__ == "__main__":
```

```
    usage, threshold = check_disk_usage("/")
```

```
    subject = f"[경고] 디스크 사용량 {usage:.1f}% 초과"
```

```
    body = f"""
```

```
<h3>⚠️ 디스크 경고</h3>
```

```
<p>현재 디스크 사용량은 <b>{usage:.1f}%</b> 입니다.<br>
```

```
서버 점검이 필요합니다.</p>
```

```
"""
```

```
    send_email(subject, body)
```

```
    print(f"⚠️ 이메일 발송 완료")
```

이메일 발송

☆ [경고] 디스크 사용량 77.0% 초과

^ 보낸사람

amugona.2606@gmail.com

받는사람

seorab@naver.com

seorab81@gmail.com

2026년 7월 26일 (일) 오후 12:32

! 디스크 경고

현재 디스크 사용량은 77.0% 입니다.
서버 점검이 필요합니다.

[경고] 디스크 사용량 77.0% 초과

받은편지함 x



amugona.2606@gmail.com

seorab, 나에게

! 디스크 경고

현재 디스크 사용량은 77.0% 입니다.
서버 점검이 필요합니다.

구글 이메일 앱비밀번호 발급

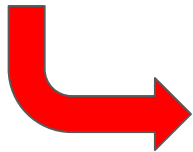
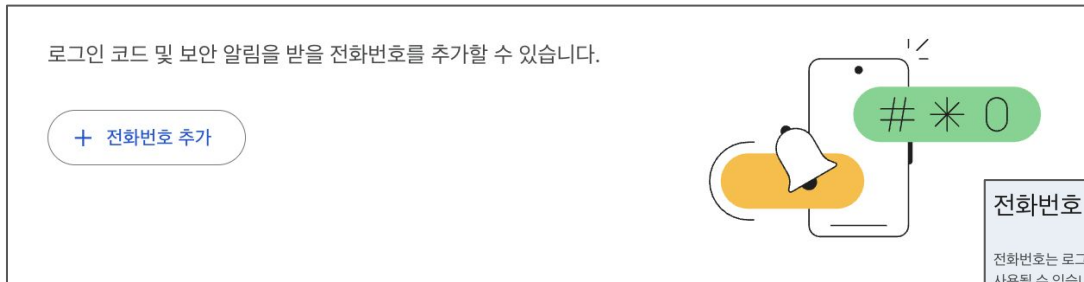
1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증]



패스키, 전화번호 등 2단계 인증 수단
추가

구글 이메일 앱비밀번호 발급

1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증 전화번호]



전화번호 추가

전화번호는 로그인 시 본인 인증에 사용되며 비정상적인 활동이 감지된 경우 알림을 받기 위해 사용될 수 있습니다.

 ▼

Google Voice 번호를 사용할 수는 있지만, Google 계정에 액세스할 수 없게 된 경우 이 번호로 코드를 받을 수 없습니다. 이동통신사 요금이 부과될 수 있습니다. [Google이 정보를 사용하는 방법 자세히 알아보기](#) ⓘ

☒ 문자 메시지로 코드 받기

☐ 음성 메시지로 코드 받기

구글 이메일 앱비밀번호 발급

1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증 사용 설정]



2단계 인증 사용 설정
로그인 시 2단계로 전화번호를 사용하여 계정 보안을 한층 강화하세요.

사용 설정

다음 번호로 로그인 코드를 받을 수 있습니다. Google 계정 복구에 사용할 수 있는 다른 번호가 더 있을 수 있습니다. [복구 전화번호 관리](#)

 010-8478-8182

🗑️

구글 이메일 앱비밀번호 발급

1. 구글 gmail.com 계정 준비 (ooo@gmail.com)
2. [Google 계정 관리] → [보안 및 로그인]
→ [2단계 인증] → [2단계 인증 사용 설정]

2단계

Google 계정에 액세스할 수 있도록 이 정보를 최신 상태로 유지하고 로그인 옵션을 추가하세요.

	패스키 및 보안 키	 패스키 1개
	Google 메시지	 기기 1대
	OTP	 OTP 앱 추가
	전화번호	 010-8478-8182

2단계 인증 사용 설정

이제 내 계정이 2단계 인증으로 보호됩니다



로그인할 때 가장 안전한 2단계를 완료하라는 메시지가 표시되므로 이 정보가 항상 최신 상태여야 합니다.

	Google 메시지	 기기 1대
	전화번호	 010-8478-8182

구글 이메일 앱비밀번호 발급

3. Google 계정 검색에서 "앱 비밀번호" 검색

또는 <https://myaccount.google.com/apppasswords> 접속

→ "앱 이름" 입력 → 16자리 비밀번호 발급

← 앱 비밀번호

앱 비밀번호를 사용하면 최신 보안 표준을 지원하지 않는 오래된 앱 및 서비스에서 Google 계정에 로그인할 수 있습니다.

앱 비밀번호는 최신 보안 표준을 사용하는 최신 앱 및 서비스보다 보안 수준이 낮습니다. 앱 비밀번호를 만들기 전에 앱에 로그인하려면 비밀번호가 필요한지 확인해야 합니다.

[자세히 알아보기](#)

앱 비밀번호가 없습니다.

앱 전용 비밀번호를 새로 만들려면 아래에 앱 이름을 입력하세요...

앱 이름

생성된 앱 비밀번호

기기용 앱 비밀번호

orzkg gmnw uhmu bgqp

사용 방법

설정하려는 애플리케이션 또는 기기의 Google 계정 설정으로 이동합니다. 비밀번호를 위에 표시된 16자리 비밀번호로 교체합니다.

일반적인 비밀번호와 마찬가지로 이 앱 비밀번호는 Google 계정에 대한 완전한 액세스 권한을 부여합니다. 비밀번호를 기억하지 않아도 되므로 적어 놓거나 다른 사용자와 공유하지 마세요.

API 서버 헬스체크 모니터링 연습문제

문제) 회사에서 운영 중인 여러 **API** 서버가 있습니다.

매일 아침 이 서버들이 잘 살아있는지 자동으로 점검하고
문제가 있는 서버가 하나라도 있으면 담당자에게 메일로 알려주는 프로그램을
완성하세요.

모두 정상이면 메일은 보내지 않고 콘솔에만 정상 메시지를 출력합니다.

[health_check_email.ipynb](#)

[참고사항]

`requests.get(url)`은 서버가 500 에러를 응답하더라도
오류가 발생되지 않고, `res.status_code`에 500이
담긴

응답 객체를 돌려줍니다.

[실행결과]

⚠ 이메일 발송 완료 (1개 서버 이상)

☆ [경고] API 헬스체크 - 1개 서버 이상 감지 📧

보낸사람 amugona.2606@gmail.com

받는사람 seorab@naver.com

2026년 7월 26일 (일) 오후 12:57

⚠ API 헬스체크 결과

총 5개 중 1개 서버에서 문제가 발견되었습니다.

- 로또 API - 상태코드 500 (응답시간 0.22초)

7.28.(화)

모듈module

1. 한 개의 파이썬 파일이 하나의 모듈이 됨
2. 변수, 함수, 클래스를 가질 수 있음 (보통 함수)
3. 파이썬을 설치할 때 기본 표준 라이브러리들을 제공해주지만
필요하다면 "pip install" 을 통해 외부 모듈을 설치하는 것도 가능
ex) pip install requests
4. 모듈을 가져오는 2가지 방법
 - 1) import 모듈명
 - 2) from 모듈명 import 클래스/함수

예외 처리

1. `except` 사용 시 메시지 확인하기 → `Exception as e`
2. 오류의 종류를 정확히 지정하기 → `OOOError as e`
3. 외부 모듈을 사용중이라면 “외부 모듈.OOOError”
4. (오류발생) 코드의 범위 확인하기
→ `try: except:` 위치 선정

* 항상 사용자를 배려해서 후속조치를 할 수 있도록 코드 작성

HTTP 메소드 (서버와의 통신 방식)

1. 9가지 종류 (실제로는 2가지를 주로 사용)

POST, GET, PUT, DELETE

PATCH, HEAD, OPTIONS, CONNECT, TRACE

2. GET: ? 기호로 요청 데이터를 전달

POST: HTTP Body로 요청 데이터를 전달

GET은 requests.get(url, params={...})

POST는 requests.post(url, data={...}) 또는 json={...}) 사용

동행복권 사이트에서 저번주 로또 1등 번호 확인하기

- 1) 목적 웹페이지로 접속
- 2) 개발자도구 열기
- 3) Network 탭에서 통신 내역 확인
- 4) GET/POST, Payload 등 요청 방식과 데이터 확인
- 5) 응답결과값 확인 - Text 인지 / JSON 인지
- 6) 데이터 파싱Parsing

동행복권 사이트에서 저번주 로또 1등 번호 확인하기

```
url = 'https://www.dhlottery.co.kr/lt645/selectPstLt645InfoNew.do'
```

```
query_params = {  
    'srchDir': 'center',  
    'srchLtEpsd': 1163,  
    '_': 1785206173568  
}
```

```
res = requests.get(url, params=query_params)
```

```
result = res.json()
```

```
num1 = result['data']['list'][0]['tm1WnNo']
```

```
num2 = result['data']['list'][0]['tm2WnNo']
```

```
num3 = result['data']['list'][0]['tm3WnNo']
```

```
num4 = result['data']['list'][0]['tm4WnNo']
```

```
num5 = result['data']['list'][0]['tm5WnNo']
```

```
num6 = result['data']['list'][0]['tm6WnNo']
```

```
print(num1, num2, num3, num4, num5, num6)
```

연습문제 - 다음 금융의 환율 정보 수집하기

회사에서 매일 아침 환율을 자동으로 확인하는 스크립트를 만들려고 합니다.

다음 금융 환율 페이지에 접속하면 화면에 실시간 환율이 표시되는데

이 데이터는 아래 **API**를 호출해서 받아옵니다.

그런데 이 **API**를 그냥 호출하면 응답을 받을 수 없습니다.

어떻게 하면 뚫을 수 있는지 요청헤더 값을 추가/수정하면서 찾아야 됩니다.

```
import requests
url = 'https://finance.daum.net/api/exchanges/summaries'
req_headers = {
    '???': '???'
}
res = requests.get(url, headers=req_headers)
print(res.status_code)
print(res.json())
```

7.29.(수)

CSS 선택자 정리

선택자 종류	문법	예시
전체	*	* { }
태그	태그명	li { }
클래스	.클래스명	.product { }
아이디	#아이디명	#gnb { }
속성	[속성=값]	[data-stock="out"] { }
인접 형제	A + B	.highlight + tr { }
일반 형제	A ~ B	.special ~ li { }
자식	A > B	.wrapper > main { }
하위	A B	.wrapper li { }
nth-child	:nth-child(n)	tr:nth-child(2) { }
nth-of-type	:nth-of-type(n)	td:nth-of-type(2) { }

CSS 선택자 연습문제 (1 / 4)

요구사항에 맞는 선택자와 **CSS** 속성을 직접 작성해보세요.
파일을 저장하고 브라우저로 열면 바로 결과를 확인할 수 있습니다.
사용하는 **CSS** 속성은 아래 몇 가지만 알면 충분합니다.

- `color` : 글자 색
- `background-color` (또는 `background`) : 배경 색
- `font-weight: bold` : 글자 굵게
- `text-align: center` : 가운데 정렬
- `border: 2px solid red` : 테두리
- `font-size: 20px` : 글자 크기

CSS 선택자 연습문제 (2 / 4)

- 1) `info-box` 클래스 안에 있는 모든 요소의 글자 크기를 `20px`로 만들어보세요.
- 2) `product-list` 아이디 안의 모든 요소에 파란색 테두리를 넣어보세요.
- 3) 페이지에 있는 모든 `li` 태그의 글자 색을 초록색으로 바꿔보세요.
- 4) 모든 `td` 태그의 텍스트를 가운데 정렬 해보세요.
- 5) `product` 클래스를 가진 요소들의 배경색을 노란색으로 바꿔보세요.
- 6) `ad` 클래스를 가진 모든 광고 영역에 빨간색 테두리를 추가해보세요.

CSS 선택자 연습문제 (3 / 4)

- 7) gnb(상단 메뉴, id="gnb")의 배경색을 검정으로 / 글자색을 흰색으로 바꿔보세요.
- 8) id="price-table" 인 표의 글자 크기를 18px로 바꿔보세요.
- 9) data-stock="out" 속성을 가진 행(품절 상품)의 배경색을 회색으로 바꿔보세요.
- 10) target="_blank" 속성을 가진 링크(a 태그)의 글자색을 초록색으로 바꿔보세요.
- 11) highlight 클래스가 붙은 행(키보드, 품절) 바로 다음 행의 배경색을 하늘색으로 바꿔보세요.
- 12) special 클래스가 붙은 상품(체리) 이후에 나오는 모든 li 상품의 글자를 굵게 만들어보세요.

CSS 선택자 연습문제 (4 / 4)

- 13) `wrapper` 클래스 바로 아래에 있는 `main` 태그의 배경색을 연한 노란색으로 바꿔보세요.
- 14) `product-list` 목록의 바로 아래 자식인 `li` 들만 글자색을 보라색으로 바꿔보세요.
- 15) 표의 본문(`tbody`) 안에서 2번째 행의 배경색을 주황색으로 바꿔보세요.
- 16) 상품 목록(`product-list`)에서 홀수 번째 `li` 들의 배경색을 연회색으로 바꿔보세요.
- 17) 각 행(`tr`)에서 2번째 `td` (가격 칸)의 글자를 굵게 만들어보세요.
- 18) 표 안에서 첫 번째 `tr`(헤더 행)의 글자색을 빨간색으로 바꿔보세요.

삼성중공업 뉴스룸 데이터 가져오기

https://www.samsungshi.com/Ko/Notice_news.aspx

```
import requests
url = "https://www.samsungshi.com/Ko/Notice_news.aspx"
headers = {
    'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36',
}
res = requests.get(url, headers=headers)
res.encoding = 'utf-8'

soup = bs(res.text, "html.parser")
lis = soup.select('li[data-aos=fade]')
for li in lis:
    print(li)
```

자격증 등 시험문제 사이트

<https://www.comcbt.com/>

<https://www.machuda.kr/learning/certification>

데이터베이스 접속 정보

host / ip: svc.sel3.cloudtype.app

username: root

password: 1234

port: 31776

db: python / 각자 아이디

데이터베이스 기본 개념

1. 데이터베이스

- 데이터를 관리하는 프로그램(서버)
- 접속 시 보이는 목록

2. 테이블

- 데이터베이스 안에 있는 개별 표

* 데이터베이스 = 엑셀 파일, 테이블 = 시트

데이터베이스의 Key

1. 기본키 (primary key)

- 데이터를 구분(식별)할 수 있는 값으로 반드시 존재해야 됨

2. 외래키 (foreign key)

- 다른 테이블에서의 기본키
- 기본키로 존재하고 있어야 입력 가능하고
역으로 외래키로 쓰여지는 기본키는 삭제 불가

그 외 대체키, 후보키, 수퍼키

테이블 간의 연관 관계

1. 1:N

- 데이터는 종류별로 나뉘고, 서로 연관관계를 맺을 수 있음
- 더 많이 등장하는 쪽에 외래키가 위치
ex) 게시물 1개 - 댓글 N개 / 영상 1개 - 좋아요 N개
- 우리 주변에서 가장 흔하게 볼 수 있는 관계

2. N:N

- 1:N 관계의 연장선
- 매핑 테이블이 필수이며
매핑 테이블과 각 방향의 테이블은 1:N 또는 N:1 관계로 구성됨
ex) 버스, 택시, 공유 킥보드 (사용자 대 이용수단)

데이터베이스에서 사용할 SQL 종류

1. 입력

INSERT INTO "테이블" (컬럼1, 컬럼2, ...) **VALUES** (값1, 값2, ...)

2. 조회

SELECT * **FROM** "테이블"

SELECT 컬럼1, 컬럼2, ... **FROM** "테이블"

3. 수정

UPDATE "테이블" **SET** 컬럼1=값1, 컬럼2=값2, ...

4. 삭제

DELETE FROM "테이블"

수정 / 삭제는 모든 데이터에 적용되므로 주의!

WHERE 조건을 이용해서 수정/삭제될 데이터 지정

7.30.(목)

[연습문제] 함수 활용 - 주차장 지역명, 지역번호 추출하기

명절 무료 주차장 데이터 사용해서 아래 조건을 만족하는 SQL 작성하기

- 1) 전화번호(tel) 에서 지역번호만 추출
- 2) 전화번호가 없는 데이터는 제외
- 3) 지역번호 오름차순으로 정렬

institution VARCHAR	sido VARCHAR	지역번호 VARCHAR	tel VARCHAR
하계중학교	서울특별시	02	02-3399-0337
신계초등학교	서울특별시	02	02-902-0008
청원초등학교	서울특별시	02	02-3399-7705
자운고등학교	서울특별시	02	02-998-2929
서울시(본청)	서울특별시	02	02-2290-6449
서울시(본청)	서울특별시	02	02-2290-6449
노곡중학교	서울특별시	02	02-996-5231
서울시(본청)	서울특별시	02	02-2290-6449
도봉중학교	서울특별시	02	02-6913-6664
서울시(본청)	서울특별시	02	02-2290-6449
방학중학교	서울특별시	02	02-956-8343
서울시(본청)	서울특별시	02	02-2290-6449
창북중학교	서울특별시	02	02-907-8244
서울시(본청)	서울특별시	02	02-2290-6449

[연습문제] 함수 활용 - 주차장 지역명, 지역번호 추출하기

명절 무료 주차장 데이터 사용해서 아래 조건을 만족하는 SQL 작성하기

```
SELECT institution, tel, sido,  
       SUBSTR(tel, 1, 3),  
       INSTR(tel, '-'),  
       SUBSTR(tel, 1, INSTR(tel, '-')-1)  
FROM holiday_parking  
WHERE tel != "  
ORDER BY SUBSTR(tel, 1, INSTR(tel, '-')-1);
```

[연습문제] 함수 활용 - 국회의원 출생 연도와 월 추출하기

국회의원 데이터를 사용해서 아래 조건을 만족하는 SQL 작성하기

- 1) 생년월일(bth_date)에서 출생 연도와 월을 추출
- 2) 생년월일 정보가 없는 데이터는 제외
- 3) 한글이름 오름차순으로 정렬

한글이름 VARCHAR	생년월일 VARCHAR	출생연도 VARCHAR	출생월 VARCHAR
갈봉근	1932-04-06	1932	04
갈봉근	1932-04-06	1932	04
강경식	1936-05-10	1936	05
강경식	1940-12-12	1940	12
강경식	1936-05-10	1936	05
강경식	1936-05-10	1936	05
강경옥	1907-11-11	1907	11
강경옥	1907-11-11	1907	11
강경옥	1907-11-11	1907	11
강근호	1934-01-25	1934	01
강금식	1941-05-29	1941	05

[연습문제] 함수 활용 - 국회의원 출생 연도와 월 추출하기

국회의원 데이터를 사용해서 아래 조건을 만족하는 SQL 작성하기

```
SELECT * FROM assembly_member;
SELECT hg_nm, sch_unit_cd,
       bth_date,
       INSTR(bth_date, '-'),
       SUBSTR(bth_date, 1, INSTR(bth_date, '-')-1),
       SUBSTR(bth_date, INSTR(bth_date, '-')+1, 2)
FROM assembly_member
WHERE bth_date IS NOT NULL
      AND bth_date != ''
ORDER BY hg_nm;
```

조금 더 수준있는 SQL 연습

https://school.programmers.co.kr/learn/challenges?tab=sql_practice_kit

통계 함수 특징

1. null은 처리하지 못함
2. 데이터를 1개로 함축
 - 여러 행(row)의 데이터를 요약하여 하나의 결과값 반환
3. GROUP BY와 자주 함께 사용
 - 전체 집계뿐 아니라 그룹별 집계도 가능

GROUP BY

1. 그룹으로 지정된 열을 **SELECT** 절에 표시
 - **GROUP BY**에 사용된 열은 결과에도 같이 보이는게 기본
2. 보통 통계 함수와 같이 사용
3. 그룹이 형성된 후 조건은 **HAVING** 사용
 - **WHERE** : 그룹이 만들어지기 전 동작
 - **HAVING** : 그룹이 만들어진 후 동작

[연습문제] GROUP BY 활용 - 교통사고 데이터

교통사고 데이터를 사용해서 아래 조건을 만족하는 SQL 작성하기

1) 요일별 교통사고 사망자 수

WEEK	VARCHAR	SUM(DIE)	DECIMAL
금		681	
목		626	
수		579	
월		610	
일		550	
토		660	
화		586	

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터

2) 지역별(시/도, 구/군/시) 교통사고 사망자 수

AREA1 VARCHAR	AREA2 VARCHAR	SUM(DIE) DECIMAL
강원	강릉시	21
강원	고성군	7
강원	동해시	9
강원	삼척시	16
강원	속초시	13
강원	양구군	5
강원	양양군	7
강원	영월군	9
강원	원주시	36
강원	인제군	7
강원	정선군	9

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터

3) 많이 발생한 사고 원인 1 ~ 5위

KIND VARCHAR	COUNT(*) BIGINT
횡단중	1078
기타	665
측면직각충돌	619
전도전복	480
진행중 추돌	436

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터

4) 교통사고가 가장 적게 발생한 요일

WEEK VARCHAR	COUNT(DIE) BIGINT
일	513

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터 풀이

교통사고 데이터를 사용해서 아래 조건을 만족하는 SQL 작성하기

1) 요일별 교통사고 사망자 수

```
SELECT WEEK, SUM(DIE)
FROM accident
GROUP BY WEEK;
```

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터 풀이

2) 지역별(시/도, 구/군/시) 교통사고 사망자 수

```
SELECT AREA1, AREA2, SUM(DIE)
FROM accident
GROUP BY AREA1, AREA2;
```

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터 풀이

3) 많이 발생한 사고 원인 1 ~ 5위

```
SELECT KIND, COUNT(*)  
FROM accident  
GROUP BY KIND  
ORDER BY COUNT(*) DESC  
LIMIT 0, 5;
```

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 교통사고 데이터 풀이

4) 교통사고가 가장 적게 발생한 요일

```
SELECT WEEK, COUNT(DIE)
FROM accident
GROUP BY WEEK
ORDER BY COUNT(DIE) ASC
LIMIT 0, 1;
```

컬럼	타입	설명
ID	INT	순번
YEAR	INT	년도
AMPM	VARCHAR(2)	주간/야간
WEEK	VARCHAR(1)	요일
DIE	INT	사망자수
AREA1	VARCHAR(10)	시/도
AREA2	VARCHAR(10)	구/군/시
KIND	VARCHAR(50)	원인(사유)

[연습문제] GROUP BY 활용 - 방문자 데이터

방문자 데이터를 사용해서 아래 조건을 만족하는 SQL 작성하기

1) 방문횟수가 2번 이상인 IP 현황

IP_ADDRESS VARCHAR	COUNT(*) BIGINT
39.7.59.172	4
220.133.3.211	4
115.140.8.102	4
211.36.137.177	4
112.162.228.113	3
110.15.215.21	3
223.39.146.204	3
125.186.105.97	3

컬럼	타입	설명
<u>VST_ID</u>	INT	순번
IP_ADDRESS	VARCHAR(20)	접속 IP
VST_PATH	VARCHAR(2)	방문경로
VST_TIME	VARCHAR(1)	접속시각

[연습문제] GROUP BY 활용 - 방문자 데이터

2) 연도별 방문자 현황

DATE_FORMAT(VST_TIME, '%Y') VARCHAR	COUNT(*) BIGINT
2017	13123
2018	9909
2019	12833
2020	1014

컬럼	타입	설명
<u>VST_ID</u>	INT	순번
IP_ADDRESS	VARCHAR(20)	접속 IP
VST_PATH	VARCHAR(2)	방문경로
VST_TIME	VARCHAR(1)	접속시각

3) 월별 방문자 현황 (방문자 수 내림차순)

DATE_FORMAT(VST_TIME, '%m') VARCHAR	COUNT(*) BIGINT
07	9756
12	3820
10	3491
08	3249
01	2572

[연습문제] GROUP BY 활용 - 방문자 데이터 풀이

방문자 데이터를 사용해서 아래 조건을 만족하는 SQL 작성하기

1) 방문횟수가 2번 이상인 IP 현황

```
SELECT IP_ADDRESS, COUNT(*)  
FROM visitor_tb  
GROUP BY IP_ADDRESS  
HAVING COUNT(*) >= 2  
ORDER BY COUNT(*) DESC;
```

컬럼	타입	설명
<u>VST_ID</u>	INT	순번
IP_ADDRESS	VARCHAR(20)	접속 IP
VST_PATH	VARCHAR(2)	방문경로
VST_TIME	VARCHAR(1)	접속시각

[연습문제] GROUP BY 활용 - 방문자 데이터 풀이

2) 연도별 방문자 현황

```
SELECT DATE_FORMAT(VST_TIME, '%Y'), COUNT(*)  
  FROM visitor_tb  
 GROUP BY DATE_FORMAT(VST_TIME, '%Y');
```

3) 월별 방문자 현황 (방문자 수 내림차순)

```
SELECT DATE_FORMAT(VST_TIME, '%m'), COUNT(*)  
  FROM visitor_tb  
 GROUP BY DATE_FORMAT(VST_TIME, '%m')  
 ORDER BY COUNT(*) DESC;
```

반려동물 (animal, owner, product)

warranty		보증서		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
id	아이디	N/A	INT	N·N
model_nm	모델명	N/A	VARCHAR(50)	NULL
period	보증기간	N/A	INT	NULL

product		물품		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
id	아이디	N/A	INT	N·N
name	물품명	N/A	VARCHAR(50)	NULL
price	가격	N/A	INT	NULL
animal_id	동물 아이디	N/A	INT	NULL
warranty_id	보증서 아이디	N/A	INT	NULL

owner		사람		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
id	아이디	N/A	INT	N·N
name	이름	N/A	VARCHAR(50)	NULL

animal		반려동물		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
id	아이디	N/A	INT	N·N
name	이름	N/A	VARCHAR(50)	NULL
age	나이	N/A	INT	NULL
owner_id	사람 아이디	N/A	INT	NULL

playground		놀이터		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
id	아이디	N/A	INT	N·N
name	놀이터명	N/A	VARCHAR(50)	NULL
address	주소	N/A	VARCHAR(50)	NULL
tel	전화번호	N/A	VARCHAR(20)	NULL

playground_animals		놀이터_반려동물		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
id	아이디	N/A	INT	N·N
playground_id	놀이터 아이디	N/A	INT	NULL
animal_id	동물 아이디	N/A	INT	NULL

반려동물 (animal, owner, product)

1. '중형견 사료' 를 사준 사람의 이름은? → 최길동
2. '얼룩이' 의 주인과 사용하고 있는 물품은? → 홍길동 소형묘 사료
3. '홍길동' 이 반려동물에게 사준 모든 물품은? →
중형묘 사료
소형묘 사료
4. 모든 반려동물의 이름과 주인의 이름을 출력하시오. →
슈슈 최길동
치치 최길동
마당이 홍길동
얼룩이 홍길동

반려동물 (animal, owner, product)

5. '슈슈'가 사용하는 물품의 이름과 가격을 출력하시오. →

목줄 5000

중형견 사료 10000

6. 각 주인이 기르는 반려동물 이름과 해당 동물이 사용하는 물품 이름을 출력하시오. →

최길동 슈슈 목줄

최길동 치치 몸통줄

최길동 슈슈 중형견 사료

최길동 치치 소형견 사료

홍길동 마당이 중형묘 사료

홍길동 얼룩이 소형묘 사료

7. '홍길동'이 기르는 모든 반려동물의 이름과 나이를 출력하시오. →

마당이 5

얼룩이 7

반려동물 (animal, owner, product)

8. '소형묘 사료'를 사용하는 동물의 이름과 주인의 이름을 출력하시오. → 얼룩이 홍길동

9. 가격이 1만 원 이상인 물품과 그 물품을 사용하는 반려동물 이름을 출력하시오. →

중형견 사료 슈슈

소형견 사료 치치

중형묘 사료 마당이

소형묘 사료 얼룩이

10. 각 주인이 사준 물품의 개수를 구하시오. (주인 이름과 개수 출력) →

최길동 4

홍길동 2

반려동물 (animal, owner, product) - 풀이

1. '중형견 사료'를 사준 사람의 이름은?

```
SELECT o.name  
FROM owner o  
JOIN animal a ON o.id = a.owner_id  
JOIN product p ON a.id = p.animal_id  
WHERE p.name = '중형견 사료';
```

2. '얼룩이'의 주인과 사용하고 있는 물품은?

```
SELECT o.name AS owner_name, p.name AS product_name  
FROM animal a  
JOIN owner o ON a.owner_id = o.id  
JOIN product p ON a.id = p.animal_id  
WHERE a.name = '얼룩이';
```


반려동물 (animal, owner, product) - 풀이

3. '홍길동'이 반려동물에게 사준 모든 물품은?

```
SELECT p.name
```

```
FROM owner o
```

```
JOIN animal a ON o.id = a.owner_id
```

```
JOIN product p ON a.id = p.animal_id
```

```
WHERE o.name = '홍길동';
```

4. 모든 반려동물의 이름과 주인의 이름을 출력하시오.

```
SELECT a.name AS animal_name, o.name AS owner_name
```

```
FROM animal a
```

```
JOIN owner o ON a.owner_id = o.id;
```

반려동물 (animal, owner, product) - 풀이

5. '슈슈'가 사용하는 물품의 이름과 가격을 출력하시오.

```
SELECT p.name, p.price
```

```
FROM animal a
```

```
JOIN product p ON a.id = p.animal_id
```

```
WHERE a.name = '슈슈';
```

6. 각 주인이 기르는 반려동물 이름과 해당 동물이 사용하는 물품 이름을 출력하시오.

```
SELECT o.name AS owner_name, a.name AS animal_name, p.name AS product_name
```

```
FROM owner o
```

```
JOIN animal a ON o.id = a.owner_id
```

```
JOIN product p ON a.id = p.animal_id;
```

반려동물 (animal, owner, product) - 풀이

7. '홍길동'이 기르는 모든 반려동물의 이름과 나이를 출력하시오.

```
SELECT a.name, a.age  
FROM owner o  
JOIN animal a ON o.id = a.owner_id  
WHERE o.name = '홍길동';
```

8. '소형묘 사료'를 사용하는 동물의 이름과 주인의 이름을 출력하시오.

```
SELECT a.name AS animal_name, o.name AS owner_name  
FROM product p  
JOIN animal a ON p.animal_id = a.id  
JOIN owner o ON a.owner_id = o.id  
WHERE p.name = '소형묘 사료';
```

반려동물 (animal, owner, product) - 풀이

9. 가격이 1만 원 이상인 물품과 그 물품을 사용하는 반려동물 이름을 출력하시오.

```
SELECT p.name AS product_name, a.name AS animal_name  
FROM product p  
JOIN animal a ON p.animal_id = a.id  
WHERE p.price >= 10000;
```

10. 각 주인이 사준 물품의 개수를 구하시오. (주인 이름과 개수 출력)

```
SELECT o.name AS owner_name, COUNT(p.id) AS product_count  
FROM owner o  
JOIN animal a ON o.id = a.owner_id  
JOIN product p ON a.id = p.animal_id  
GROUP BY o.name;
```

company

job_history		인사이동		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ JOB_HISTORY_ID	인사이동아이디	N/A	INT	N·N
➡ EMPLOYEE_ID	사원번호	N/A	INT	NULL
START_DATE	근무시작일	N/A	DATE	NULL
END_DATE	근무종료일	N/A	DATE	NULL
➡ JOB_ID	아이디	N/A	VARCHAR(10)	NULL
➡ DEPARTMENT_ID	부서번호	N/A	INT	NULL

jobs		직책연봉정보		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ JOB_ID	아이디	N/A	VARCHAR(10)	N·N
JOB_TITLE	직책명	N/A	VARCHAR(35)	NULL
MIN_SALARY	최저월급	N/A	INT	NULL
MAX_SALARY	최대월급	N/A	INT	NULL

departments		부서		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ DEPARTMENT_ID	부서번호	N/A	INT	N·N
DEPARTMENT_NAME	부서명	N/A	VARCHAR(30)	NULL
➡ LOCATION_ID	부서위치아이디	N/A	INT	NULL

employees		사원		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ EMPLOYEE_ID	사원번호	N/A	INT	N·N
FIRST_NAME	이름	N/A	VARCHAR(20)	NULL
LAST_NAME	성	N/A	VARCHAR(25)	NULL
EMAIL	이메일	N/A	VARCHAR(25)	NULL
PHONE_NUMBER	전화번호	N/A	VARCHAR(20)	NULL
HIRE_DATE	입사일자	N/A	DATE	NULL
➡ JOB_ID	아이디	N/A	VARCHAR(10)	NULL
SALARY	월급	N/A	INT	NULL
COMMISSION_PCT	판매 수수료 비율	N/A	INT	NULL
➡ MANAGER_ID	관리자 사원번호	N/A	INT	NULL
➡ DEPARTMENT_ID	부서번호	N/A	INT	NULL

locations		부서위치		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ LOCATION_ID	부서위치아이디	N/A	INT	N·N
STREET_ADDRESS	주소	N/A	VARCHAR(40)	NULL
POSTAL_CODE	우편번호	N/A	VARCHAR(12)	NULL
CITY	도시	N/A	VARCHAR(30)	NULL
STATE_PROVINCE	주	N/A	VARCHAR(25)	NULL
➡ COUNTRY_ID	국가아이디	N/A	CHAR(2)	NULL

countries		국가		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ COUNTRY_ID	국가아이디	N/A	CHAR(2)	N·N
COUNTRY_NAME	국가명	N/A	VARCHAR(40)	NULL
➡ REGION_ID	지역아이디	N/A	INT	NULL

regions		지역(대륙)		
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용
➡ REGION_ID	지역아이디	N/A	INT	N·N
REGION_NAME	지역명	N/A	VARCHAR(25)	NULL

company

1. 사원번호가 **140**인 사원의 성과 이름, 근무부서가 위치한 나라명을 확인하시오 →
Patel Joshua Shipping United States of America
2. 'IT' 부서에 근무하는 사원 수와 모든 사원의 급여 합을 확인하시오 →
5 28800
3. 'Asia' 지역(대륙)에 속해있는 국가를 확인하시오 →
Australia
China
India
Japan
Malaysia
Singapore

company

4. 'Washington' 주에 위치한 부서 목록을 확인하시오 →

Administration

Purchasing

...

5. 부서별 사원 수를 구하시오 (부서명, 인원수) →

IT	5
----	---

Shipping	45
----------	----

Sales	34
-------	----

...

6. 국가별로 근무하는 사원 수를 구하시오 (국가명, 사원수) →

United States of America	68
--------------------------	----

United Kingdom	35
----------------	----

Canada	2
--------	---

Germany	1
---------	---

company

7. 지역(Region)별 사원 수와 평균 급여를 구하시오 →

Americas	70	5192
Europe	36	8917

Outer Join 연습문제

- 1) BOARD 테이블 생성 및 데이터 입력
- 2) BOARD_CODE 테이블 생성 및 데이터 입력
- 3) BOARD_FILE 테이블 생성 및 데이터 입력

문제 1) 가 ~ 다

문제 2) 가 ~ 다

7.31.(금)

데이터 조회

1. 기본 형태: **select** 컬럼 **from** 테이블명

- **distinct**: 중복 제거
- **as**: 별칭 (컬럼명을 내 마음대로)
- **+ - * /**: 산술연산

2. **where**: 조건 검색

- 비교연산자: **= > < >= <= !=**
- 논리연산자: **and or not**
- 포함되는 내용 검색: **like + %%**

데이터 조회

3. order by: 정렬

- 오름차순: **asc**
- 내림차순: **desc**

4. limit: 개수 제한 (데이터 끊어주기)

5. 함수: 숫자, 문자, 날짜 등

- 숫자: 연산 관련
- 문자: 찾기, 변경, 잘라내기 등
- 날짜: 현재일시, 출력양식 지정

데이터 조회

6. 통계(집계) 함수

- 개수, 평균, 합계, 최대, 최소
- 여러개의 데이터를 하나의 형태로 통합
- null 값은 무시

7. group by: 그룹 나누기 (같은 종류로 묶기)

having: 그룹을 나눈 이후 조건 검색

8. join: 테이블 연결하기 (여러 개의 테이블을 하나로 합치기)

- 보통 한 테이블의 기본키와 다른 테이블의 외래키를 연결
- 여러 개를 이어 붙여 큰 테이블이 되었을뿐 특별히 달라지는 부분은 없음
(where / group by / order by 등 기존 순서와 흐름은 그대로)

조회 SQL 실행 순서

SELECT (5)

FROM (1)

JOIN -- LEFT JOIN 또는 RIGHT JOIN 가능
ON

WHERE (2)

GROUP BY (3)

HAVING (4)

ORDER BY (6)

LIMIT (7)

Sub Query

select 안에 또 다른 **select**를 작성하는 방법

1. () 소괄호 사용
2. **order by**와 **limit**을 제외한 나머지 절에 사용
3. 여러개의 행이 반환되는 경우 **in** 연산자 사용

ex) **select * from emp**

where sal in (select sal from emp where deptno=20)

※ 데이터베이스 성능에 영향을 줄 수 있음 (성능 저하)

국회의원 명단 데이터를 JSON 형태로 응답하기 (요구사항 해결 전)

```
@app.route('/assembly')
def assembly():
    import pymysql
    conn = pymysql.connect(
        host='svc.sel3.cloudtype.app', user='root',
        password='1234',
        db='ggoreb', charset='utf8', port=31776
    )
    cursor = conn.cursor()
    sql = """
        select * from assembly_member
        limit 0, 10
    """
    cursor.execute(sql)
    result = cursor.fetchall()

    cursor.close()
    conn.close()

    return result
```


국회의원 명단 데이터를 JSON 형태로 응답하기 (해결)

```
@app.route('/assembly')
def assembly():
    import pymysql

    conn = pymysql.connect(
        host='svc.sel3.cloudtype.app', user='root',
        password='1234',
        db='ggoreb', charset='utf8', port=31776
    )
    cursor = conn.cursor()
    sql = '''
        select * from assembly_member
        limit 0, 10
    '''
    cursor.execute(sql)
    result = cursor.fetchall()
    print(result)
    # result2 = [ list(r) for r in result ]
    result2 = jsonify(result)

    cursor.close()
    conn.close()

    return result2
```

고객 데이터 조회 (customer 함수)

```
@app.route('/customers')
def customers():
    import pymysql
    from pymysql.cursors import DictCursor

    conn = pymysql.connect(
        host='svc.sel3.cloudtype.app', user='root',
        password='1234',
        db='ggoreb', charset='utf8', port=31776
    )
    cursor = conn.cursor(DictCursor)
    sql = '''
        select * from 고객
    '''
    cursor.execute(sql)
    result = cursor.fetchall()

    cursor.close()
    conn.close()
    return render_template('customers.html', data=result)
```

고객 데이터 조회 (customer.html)

```
DOCTYPE html">
<html lang="ko">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0" >
<title>고객 센터 대시보드</title>
<!-- Tailwind CSS CDN -->
<script src="https://cdn.tailwindcss.com" ></script>
<!-- FontAwesome 아이콘 -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" >
<!-- Moto Sans KR 폰트 -->
<link href="https://fonts.googleapis.com/css2?family=Moto+SansKR:wght@300;400;500;600;700&display=swap" rel="stylesheet">
<style>
body { font-family: 'Moto Sans KR', sans-serif; }
</style>
</head>
<body class="bg-slate-50 min-h-screen text-slate-800 p-4 md:p-8" >
<div class="max-w-7xl mx-auto space-y-6">
<!-- 헤더 영역 -->
<div class="flex flex-col md:flex-row md:items-center justify-between gap-4 bg-white p-6 rounded-2xl shadow-sm border border-slate-100" >
<div>
<h1 class="text-2xl font-bold text-slate-900 flex items-center gap-2" >
<span class="fa-solid fa-users text-indigo-600" ></!> 고객 정보 관리
</h1>
<p class="text-slate-500 text-sm mt-1">등록된 현재 고객의 상세 정보 및 직원들 현황입니다.</p>
</div>
<div>
<!-- 검색창 -->
<div class="relative w-full md:w-72" >
<div class="fa-solid fa-magnifying-glass absolute left-3.5 top-1/2 -translate-y-1/2 text-slate-400" ></!>
<input type="text" id="searchInput" onkeyup="filterTable()" placeholder="이름, 아이디, 직업 입력..." />
<div class="w-full pl-10 pr-4 py-2 bg-slate-50 border border-slate-200 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20 focus:border-indigo-500 transition-all" >
</div>
</div>
</div>
<!-- 테이블 요약 통계 카드 영역 -->
<div class="grid grid-cols-4 md:grid-cols-3 gap-4">
<!-- 총 고객 수 -->
<div class="bg-white p-5 rounded-2xl border border-slate-100 shadow-sm flex items-center gap-4" >
<div class="w-12 h-12 rounded-xl bg-indigo-50 text-indigo-600 flex items-center justify-center text-xl font-bold" >
<div class="fa-solid fa-user-group" ></!>
</div>
<div>
<p class="text-xs text-slate-400 font-medium">총 고객 수</p>
<p class="text-xl font-bold text-slate-800">{ total_count } 명</p>
</div>
</div>
<!-- 총 직원들 수직 (None 제외의 인원 처리) -->
<div class="bg-white p-5 rounded-2xl border border-slate-100 shadow-sm flex items-center gap-4" >
<div class="w-12 h-12 rounded-xl bg-emerald-50 text-emerald-600 flex items-center justify-center text-xl font-bold" >
<div class="fa-solid fa-coins" ></!>
</div>
<div>
<p class="text-xs text-slate-400 font-medium">총 직원들 수직</p>
<p class="text-xl font-bold text-slate-800">{{ ("{}").format(total_points) }} P</p>
</div>
</div>
<!-- 평균 나이 (None 제외 및 0 나누기 방지) -->
<div class="bg-white p-5 rounded-2xl border border-slate-100 shadow-sm flex items-center gap-4" >
<div class="w-12 h-12 rounded-xl bg-amber-50 text-amber-600 flex items-center justify-center text-xl font-bold" >
<div class="fa-solid fa-cake-candles" ></!>
</div>
<div>
<p class="text-xs text-slate-400 font-medium">고객 평균 나이</p>
<p class="text-xl font-bold text-slate-800">{{ avg_age | round(1) }} 세</p>
</div>
</div>
</div>
<!-- 테이블 영역 -->
<div class="bg-white rounded-2xl shadow-sm border border-slate-100 overflow-hidden" >
<div class="text-sm font-weight="bold">
<table class="w-full text-left border-collapse" id="customerTable">
```

국회의원 명단 데이터 조회 (assembly2 함수)

```
@app.route('/assembly2')
def assembly2():
    import pymysql
    from pymysql.cursors import DictCursor

    conn = pymysql.connect(
        host='svc.sel3.cloudtype.app', user='root',
        password='1234',
        db='ggoreb', charset='utf8', port=31776
    )
    cursor = conn.cursor(DictCursor)
    sql = '''
        select * from assembly_member
        order by id desc
        limit 0, 100
    '''
    cursor.execute(sql)
    result = cursor.fetchall()

    cursor.close()
    conn.close()

    return render_template('assembly.html', data=result)
```

국회의원 명단 데이터 조회 (assembly html)

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0" >
  <title>대한민국 국회의원 정보 데이터</title>
  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com" ></script>
  <!-- Fontawesome 아이콘 -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" >
  <!-- Moto Sans KR 폰트 -->
  <link href="https://fonts.googleapis.com/css2?family=Moto+SansKR:wght@300;400;500;600;700&display=swap" rel="stylesheet">
  <style>
    body { font-family: 'Moto Sans KR', sans-serif; }
  </style>
</head>
<body class="bg-slate-100 min-h-screen text-slate-800 p-4 md:p-8">

  <div class="max-w-7xl mx-auto space-y-8">

    <!-- Header Banner -->
    <div class="bg-gradient-to-r from-slate-900 via-indigo-950 to-slate-900 text-white p-8 md:p-8 rounded-2xl shadow-xl border border-slate-800 flex flex-col md:flex-row items-start md:items-center justify-between gap-8">
      <div>
        <div class="h1:line-flex items-center gap-2 px-3 py-1 rounded-full bg-indigo-500/20 text-indigo-300 border border-indigo-500/30 text-xs font-semibold mb-3">
          <i class="fa-solid fa-building-columns"> </i> 대한민국 국회
        </div>
        <h1 class="text-2xl md:text-3xl font-watrabold tracking-tight">국회원 현황 정보 현황</h1>
        <p class="text-slate-400 text-sm mt-1">>국원명, 정당, 지역구, 당선 횟수 및 소속 위원회 정보를 한눈에 확인하세요.</p>
      </div>

      <!-- 검색창 -->
      <div class="relative w-full md:w-80">
        <i class="fa-solid fa-magnifying-glass absolute left-4 top-1/2 -translate-y-1/2 text-slate-400"> </i>
        <input type="text" id="searchInput" onkeyup="filterCards()" placeholder="국원명, 정당, 지역구 검색..." class="w-full pl-11 pr-4 py-3 bg-slate-800/80 border border-slate-700/80 text-white placeholder-slate-400 rounded-2xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-600 transition-all shadow-inner">
      </div>

    </div>

    <!-- 요약 통계 정보 (line2 데이터 연산 + None 오류 방지) -->
    <{ set total_count = data | length }>
    <{ set valid_ages = data | map(attribute="age") | select("number") | list }>
    <{ set age_count = valid_ages | length }>
    <{ set avg_age = (valid_ages | sum / age_count) if age_count > 0 else 0 }>

    <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
      <div class="bg-white p-5 rounded-2xl border border-slate-200/80 shadow-sm flex items-center gap-4">
        <div class="w-12 h-12 rounded-xl bg-indigo-50 text-indigo-600 flex items-center justify-center text-xl font-bold">
          <i class="fa-solid fa-user-circle"> </i>
        </div>
        <p class="text-xs text-slate-400 font-medium">>국회원 인원 수</p>
        <p class="text-xl font-bold text-slate-800"><{ total_count }> 명</p>
      </div>

      <div class="bg-white p-5 rounded-2xl border border-slate-200/80 shadow-sm flex items-center gap-4">
        <div class="w-12 h-12 rounded-xl bg-blue-50 text-blue-600 flex items-center justify-center text-xl font-bold">
          <i class="fa-solid fa-cake-candles"> </i>
        </div>
        <p class="text-xs text-slate-400 font-medium">>평균 연령</p>
        <p class="text-xl font-bold text-slate-800"><{ avg_age | round(1) }> 세</p>
      </div>

      <div class="bg-white p-5 rounded-2xl border border-slate-200/80 shadow-sm flex items-center gap-4">
        <div class="w-12 h-12 rounded-xl bg-amber-50 text-amber-600 flex items-center justify-center text-xl font-bold">
          <i class="fa-solid fa-award"> </i>
        </div>
        <p class="text-xs text-slate-400 font-medium">>초선 의원 비율</p>
        <{ set new_list = data | map(attribute="term1_year") | list }>
        <{ set first_term = new_list | select("equally") | count } | list | length }>
        <p class="text-xl font-bold text-slate-800">
          <{ if total_count > 0 }>
          <{ ((first_term / total_count) * 100) | round(0) | int }> %
        <{ else }>
          0%
        <{ endif }>
      </p>
    </div>

    <div class="bg-white p-5 rounded-2xl border border-slate-200/80 shadow-sm flex items-center gap-4">
      <div class="w-12 h-12 rounded-xl bg-amber-50 text-amber-600 flex items-center justify-center text-xl font-bold">
        <i class="fa-solid fa-lamp"> </i>
      </div>
      <p class="text-xs text-slate-400 font-medium">>국회원 정보</p>
    </div>

  </div>
```

Router 함수의 응답방법

1. 문자 반환 → HTML

ex) return '<h1>Hello</h1>'

2. 리스트 또는 딕셔너리 반환 → JSON

ex) return [1, 2, 3, ...]

return { 'name': 'kim', 'age': 30, ... }

3. render_template 반환 → HTML 파일

ex) return render_template('home.html')

user 테이블 DDL

```
CREATE TABLE user (  
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '고유 식별자(자동증가)',  
    user_id VARCHAR(255) NOT NULL UNIQUE COMMENT '이메일(로그인 ID)',  
    user_name VARCHAR(100) NOT NULL COMMENT '사용자 이름',  
    user_pw VARCHAR(255) NOT NULL COMMENT '암호화된 비밀번호',  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '가입 일자'  
);
```

회원가입 /join 라우터 함수 코드

```
@app.route("/join", methods=['GET', 'POST'])
def join():
    if request.method == 'GET':
        return render_template('join.html')
    else :
        form = request.form
        user_id = form['user_id']
        user_pw = form['user_pw']
        user_name = form['user_name']

import pymysql
from pymysql.cursors import DictCursor

conn = pymysql.connect(
    host='svc.sel3.cloudtype.app', user='root',
    password='1234',
    db='ggoreb', charset='utf8', port=31776
)

cursor = conn.cursor(DictCursor)
```


문자열 해시 (비밀번호 암호화)

```
import hashlib
```

```
old_pwd = '1234'
```

```
m = hashlib.sha256()
```

```
m.update(old_pwd.encode())
```

```
new_pwd = m.hexdigest()
```

```
# new_pwd = hashlib.sha256(old_pwd.encode()).hexdigest()
```

```
print(new_pwd)
```